

课程编号: 0008190



计算机硬件类综合性课程设计 设计报告

(2025-2026-2)

题 目: 基于 PYNQ-Z1 的驾驶员疲劳检测系统

学 院: 计算机学院

专业班级: 计算机科学与技术(实验班) 230710

报 告 人: 23071005 侯景祺, 23071031 李佳萌

23071023 康昊轩

评语:

教师签字

评阅日期

2026.6.

一、概述

1.1 选题意义

疲劳驾驶是道路交通安全中的典型风险因素之一。驾驶员在长时间驾驶、夜间驾驶或注意力下降时，眼睛闭合时间和闭眼比例会出现明显变化。相比只依赖人工观察或单一阈值判断，基于图像的疲劳检测系统能够持续采集驾驶员面部状态，并给出实时提醒，具有较强的实际应用意义。

本系统选择 PYNQ-Z1 作为实现平台，利用 Zynq 器件中 PS 与 PL 协同工作的特点，将计算量稳定、结构清晰的特征提取和 SVM 分类放入 PL 端，将 Dlib 人脸/眼部定位、状态机控制和外设提示放在 PS 端，从而在有限算力上形成可运行的疲劳检测通路。

1.2 作品功能概述

1. 实时获取摄像头画面，并通过 Dlib 低频更新人脸及眼部 ROI。
2. EyeFeature IP 在 PL 端对左右眼 ROI 进行灰度统计和结构统计，输出 8 维眼部图像特征。
3. SVM IP 接收 15 帧 EyeFeature 组成的 120D 窗口，输出判断结果 pred 和分类分数 score。
4. PS 端通过 10 s 滑动窗口统计闭眼比例，判断是否进入疲劳警报状态。
5. 系统支持 OLED 状态显示、LED/TTS 警报输出，以及无人脸时的节能待机和自动恢复检测。

1.3 所采用的平台及模块

所采用的平台：PYNQ-Z1 板卡 / Zynq-7020

所采用的模块：OV7670 摄像头

TTS 语音合成模块

2.42 寸 OLED 屏幕

RGB LED 灯珠

RJ45 网线

1.4 成员分工

侯景祺：

- (1) EyeFeature 图像特征提取 IP 设计
- (2) SVM 分类 IP 设计
- (3) SVM 数据集构建与权重定点化
- (4) Dlib 异步检测与 ROI 复用
- (5) 疲劳滑动窗口判决与待机节能模式

李佳萌：

- (1) OV7670 摄像头 IP 设计
- (2) 摄像头采集链路与 VDMA 接入
- (3) 视频录制软件实现
- (4) 画面旋转、缩放等基础处理

康昊轩：

- (1) OLED 屏幕 SPI IP 设计
- (2) RGB LED 灯珠控制模块设计
- (3) UART TTS 语音播报模块设计
- (4) OLED、LED、TTS 软件驱动设计
- (5) 系统状态显示与声光提醒实现

二、总体设计

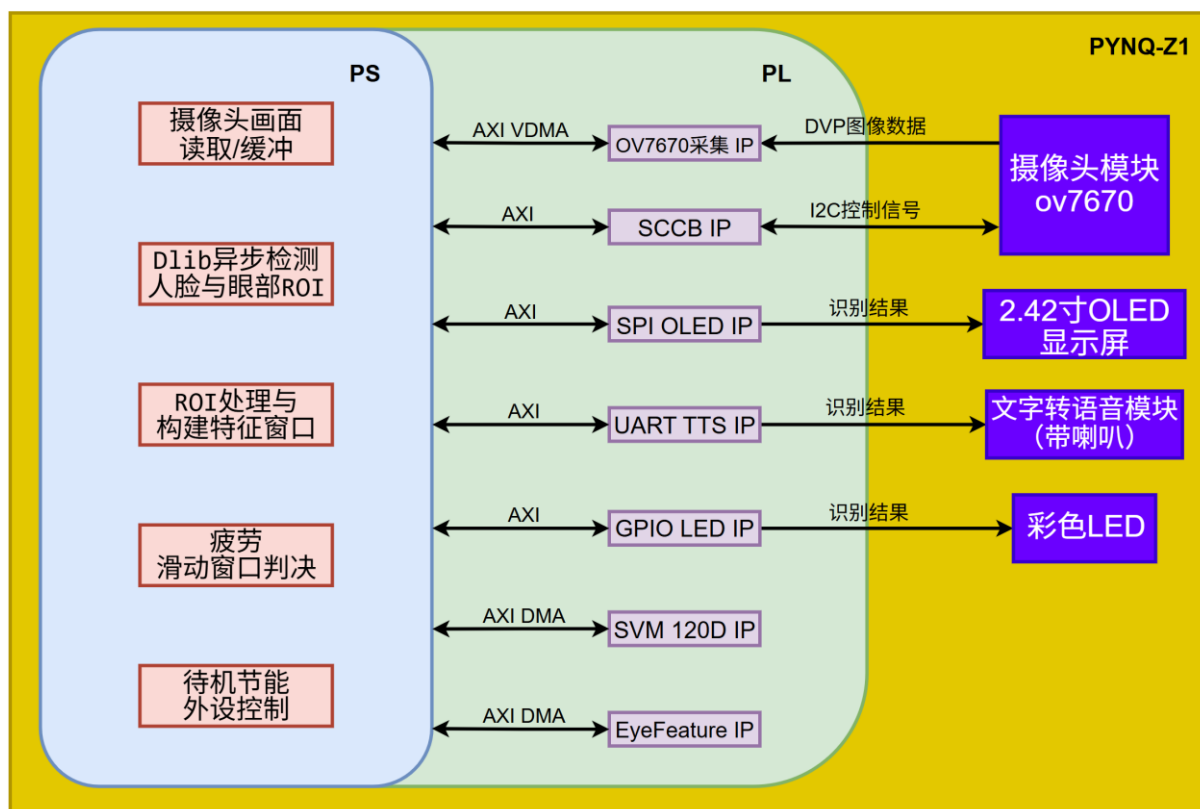


图 2.1 系统框图

本系统为基于 PYNQ-Z1 的驾驶员疲劳检测系统。

硬件方面：

采用 PYNQ-Z1 开发板作为主控平台，板载 Xilinx Zynq-7000 系列 FPGA 与双核 ARM Cortex-A9 处理器，可充分利用 PS 端软件调度能力和 PL 端硬件加速能力。系统硬件模块及 IP 设计包括：

1. OV7670 摄像头采集模块 —— 通过摄像头获取驾驶员面部图像，并经 VDMA 将画面数据传回 PS 端。
2. SCCB 摄像头配置模块 —— 通过寄存器配置摄像头工作模式、分辨率和图像输出格式。
3. EyeFeature 图像特征提取模块 —— 根据 PS 端写入的眼部 ROI，在 PL 端完成灰度统计、结构统计，并输出 8 维眼部图像特征。
4. SVM 120D 分类模块 —— 接收连续 15 帧 EyeFeature 组成的 120 维特征窗口，输出闭眼判断结果和分类分数。
5. SPI-OLED 显示模块 —— 显示系统运行状态、帧率、SVM 分数、疲劳比例等信息。
6. UART-TTS 语音播报模块 —— 在疲劳警报触发时进行语音提示。
7. GPIO-LED 模块 —— 通过 RGB LED 显示系统正常、警报等状态。

软件方面：

计算机硬件类综合性课程设计

系统软件运行在 PYNQ-Z1 的 PS 端，主要负责 bitstream 加载、各 IP 初始化、摄像头画面读取、dlib 人脸与眼部 ROI 检测、特征窗口维护、疲劳状态判决以及外设控制。

本软件运行依赖 PYNQ 板卡进入 Ubuntu 系统。通过 ssh+网线或者 USB 串口链接板卡，进入终端，可以启动 python 程序。

程序启动后，首先初始化摄像头采集链路、EyeFeature IP、SVM IP、OLED、LED 和 TTS 模块；随后进入主循环，持续读取摄像头画面，并按设定间隔更新人脸与眼部 ROI。

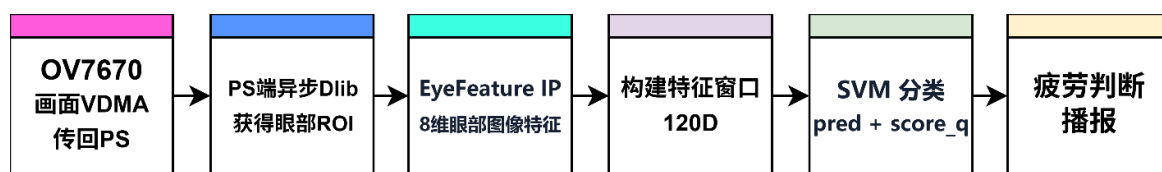


图 2.2 系统总体软硬件链路

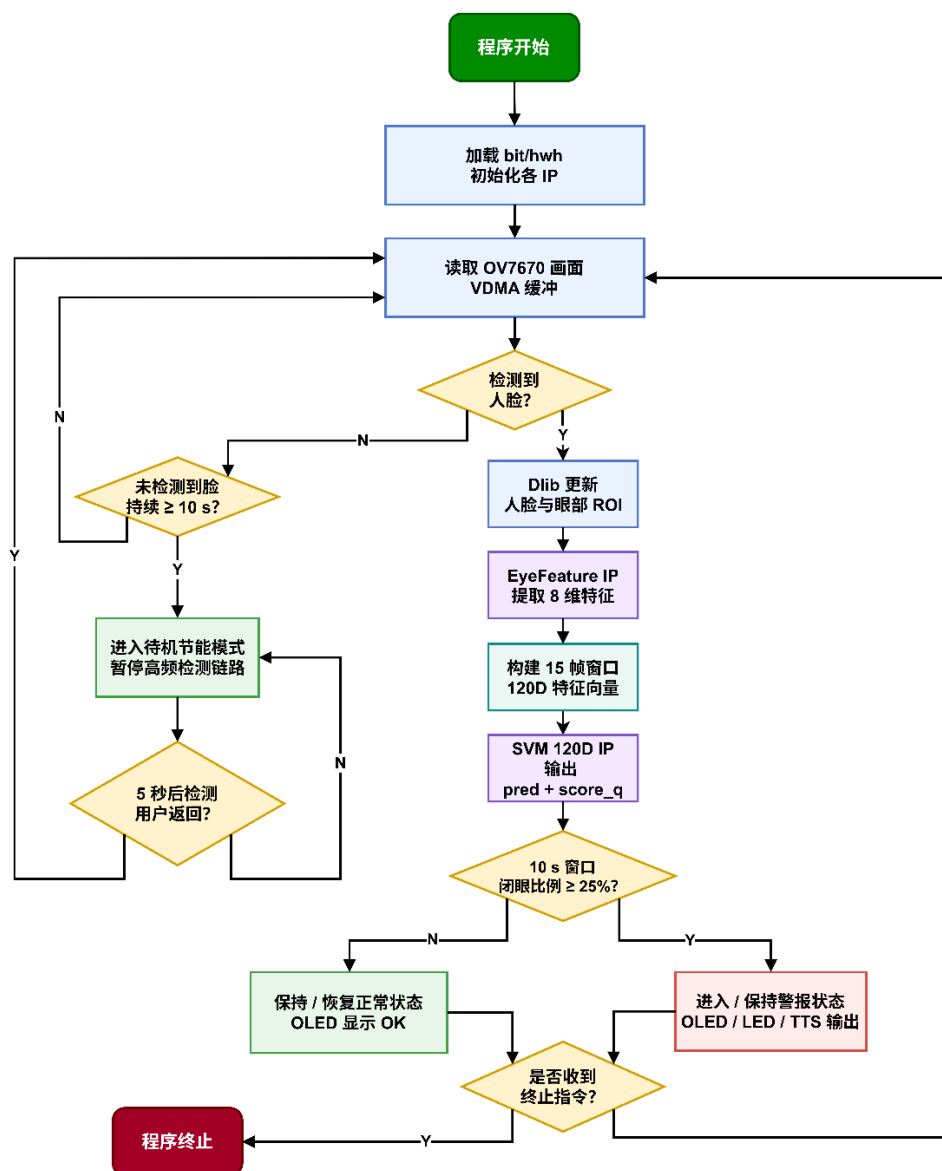


图 2.3 系统软件逻辑图

计算机硬件类综合性课程设计

在检测过程中，PS 端将最新 ROI 坐标写入 EyeFeature IP，PL 端根据 ROI 对当前帧眼部区域提取 8 维 EyeFeature。PS 端维护连续 15 帧特征窗口，并将其组成 120 维向量送入 SVM IP 完成闭眼分类。SVM 输出的 pred 和 score 进入疲劳判决逻辑，系统根据 10 秒滑动窗口内闭眼样本比例判断是否进入疲劳警报状态。若检测到疲劳，系统通过 OLED、RGB LED 和 TTS 模块进行状态显示与声光提醒；若连续一段时间未检测到人脸，则进入待机节能模式，并周期性检测用户是否返回。

三、详细设计

3.1 EyeFeature 图像特征提取模块的设计（撰写人：侯景祺）

3.1.1 IP 设计

（1）设计目标与 IP 功能

EyeFeature IP 的输入是一帧来自摄像头链路的图像流，以及 PS 端写入的左右眼 ROI 坐标；输出是当前帧对应的 8 维眼部图像特征、有效标志、ROI 版本号和调试阈值。该 IP 将灰度化、ROI 裁剪、局部直方图统计、自适应阈值、暗区结构测量和双眼融合放到 PL 端执行。

设计目标可以概括为三点。第一，避免在 PS 端对每一帧运行较重的图像处理逻辑；第二，使特征与全局亮度解耦，尽量依赖局部对比度和眼部暗区结构；第三，输出定长、定点、顺序固定的特征，便于后续 SVM IP 进行硬件分类。

- 输入对象：完整图像流、左右眼 ROI、像素格式、阈值百分位参数、定点缩放参数。
- 输出对象：8 个用于分类的定点 EyeFeature、valid、roi_version、debug_thresholds。
- 设计约束：保持顶层函数名 eye_feature 和 AXI-Lite/AXI-Stream 接口形式，便于替换原有 block design 中的同名 IP。

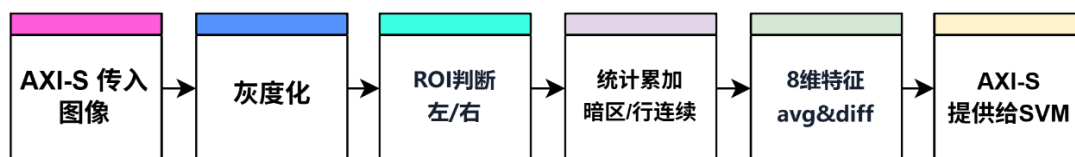


图 3.1-1 EyeFeature IP 内部处理流程

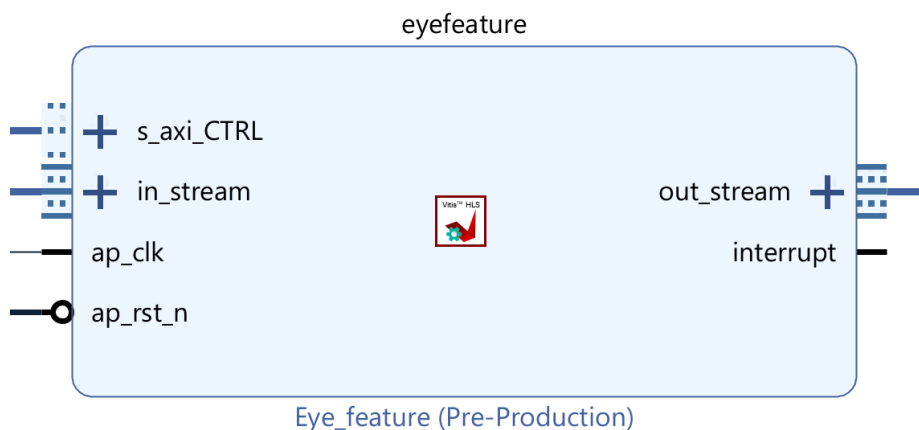


图 3.1-2 EyeFeature 在 Vivado Block Design 中的模块图

EyeFeature 的主要功能不是输出处理后的画面，而是把眼部灰度结构转化为 8 个稳定特征，并通过 AXI-Stream 送入后续 120D SVM 分类链路。

（2）眼部图像特征的理论依据

驾驶员眼部状态会改变眼部 ROI 内的灰度结构。睁眼时，瞳孔、虹膜和眼裂区域通常形成较连续的暗色结构；闭眼时，眼裂暗区的形态和连续性发生变化，局部暗像素比例、竖向

连续暗像素长度、有效暗行长度等统计量会随之改变。EyeFeature 采用结构统计而非单一灰度阈值，可以减少绝对亮度、曝光和眼镜反光对分类的影响。

由于摄像头自动曝光、室内外光照和佩戴眼镜都会改变整体灰度分布，如果直接使用固定灰度阈值，则同一眼部状态在不同光照下可能落入不同的数值范围。因此本模块使用 ROI 内部直方图计算百分位阈值，将阈值定义在局部灰度分布上。

$$Hist[b] = \{(x, y) \mid |Y(x, y)/4| = b\}, \quad b = 0, 1, \dots, 63$$

$$q_p = \min\{t \mid cumulative_{hist}(t) \geq p\% \times N\}$$

其中 N 为当前眼部内部 ROI 的像素数， q_p 表示 p 百分位灰度。代码默认使用 $low_{pct} = 10$ 和 $high_{pct} = 60$ ，对应较暗部分和局部参考亮度。之后根据两个百分位的差值计算局部对比度。

$$contrast = q_{high} - q_{low}$$

$$t_{low} = q_{low} + contrast/4,$$

$$t_{high} = q_{low} + contrast/2$$

当 $contrast$ 小于 $MIN_CONTRAST$ 时，说明当前 ROI 中局部灰度层次不足，可能是过曝、欠曝或有效纹理很弱。此时模块将结构性特征置 0，以避免把无意义的灰度噪声误认为眼部结构。

(3) 工作原理：输入像素格式与灰度化设计

摄像头链路和本地训练脚本中曾使用不同的颜色顺序，因此 EyeFeature 在硬件中保留 `pixel_format` 控制寄存器，支持灰度、RGB565 等。这样可以在不重新封装 IP 的情况下适配图像链路颜色格式。

灰度化采用整数近似公式，权重来源于常用亮度转换关系 $Y = 0.299R + 0.587G + 0.114B$ 。代码中使用 77、150、29 这组 8 bit 缩放系数，并在右移前加 128 完成四舍五入。

$$Y = (77R + 150G + 29B + 128) \gg 8$$

HLS 中 cpp 代码实现如下：

```
1. static ap_uint<8> weighted_gray(ap_uint<8> r, ap_uint<8> g, ap_uint<8> b) {
2.     #pragma HLS INLINE      ap_uint<18> y = 77 * r + 150 * g + 29 * b + 128;
3.     return (ap_uint<8>)(y >> 8);
4. }
```

表 3.1-1 EyeFeature 支持的像素格式

pixel_format	输入格式	灰度化处理方式
0	Gray8	直接取 data[7:0]，用于 C 仿真和灰度输入
1	RGB565	按 R[15:11]、G[10:5]、B[4:0] 扩展到 8 bit 后加权

2	RGB888	按 data[23:16]、data[15:8]、data[7:0] 加权
3	GRB565	适配板端 GRB 顺序，将 G/R 字段交换后加权

(4) 工作原理：ROI 裁剪与内部有效区域

PS 端 dlib 给出的眼部 ROI 往往包含一定的边缘区域，例如眉毛、眼镜框、皮肤和背景。为了降低这些区域对特征统计的影响，HLS 代码中的 `make_eye_cfg()` 不直接使用完整 ROI，而是从中截取内部区域。默认内部区域为横向 10%~90%、纵向 18%~88%。

$$x'_0 = x + \text{round}(0.10w), \quad x'_1 = x + \text{round}(0.90w)$$

$$y'_0 = y + \text{round}(0.18h), \quad y'_1 = y + \text{round}(0.88h)$$

内部 ROI 的尺寸受 `MAX_CENTER_W=128` 和 `MAX_CENTER_H=64` 限制。该限制让左右眼像素缓冲可以使用固定大小 BRAM 实现，避免动态内存，也方便 HLS 推断片上 RAM。若 ROI 越界、尺寸非法或超过限制，则该眼有效标志置 `false`。

HLS 代码实现如下：

```
1. int ix0 = round_ratio(rw, INNER_X0_NUM, INNER_DEN);
2. int ix1 = round_ratio(rw, INNER_X1_NUM, INNER_DEN);
3. int iy0 = round_ratio(rh, INNER_Y0_NUM, INNER_DEN);
4. int iy1 = round_ratio(rh, INNER_Y1_NUM, INNER_DEN);
5. if (cw <= 0 || ch <= 0 || cw > MAX_CENTER_W || ch > MAX_CENTER_H)
6.     return invalid_cfg;
```

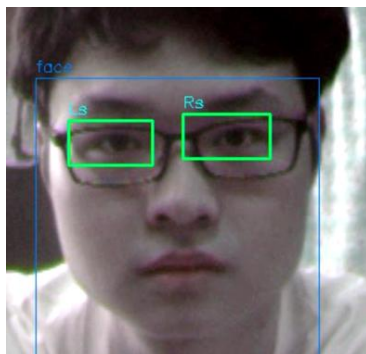


图 3.1-3 眼部 ROI 与内部统计区域示意图

(5) 工作原理：局部直方图与自适应阈值

EyeFeature 在扫描整帧图像时，只把落入左右眼内部 ROI 的灰度像素写入 `pix0/pix1` 缓冲，同时更新 `hist0/hist1`。直方图使用 64 个 bin，每个 bin 覆盖 4 个灰度级，既能保留足够的灰度分布信息，又能降低硬件存储和循环开销。

HLS 代码如下：

```
1. int bin = (int)(gray >> HIST_SHIFT); // HIST_SHIFT = 2, 64 bins
2. hist0[bin] = hist0[bin] + 1;
```

阈值计算函数 `percentile_threshold()` 遍历直方图累计像素数，当累计数首次达到目标百分位时确定对应灰度。该过程对整体亮度平移具有较好的适应性，因为阈值由当前 ROI 的灰

度排序确定，而不是由固定亮度常数确定。

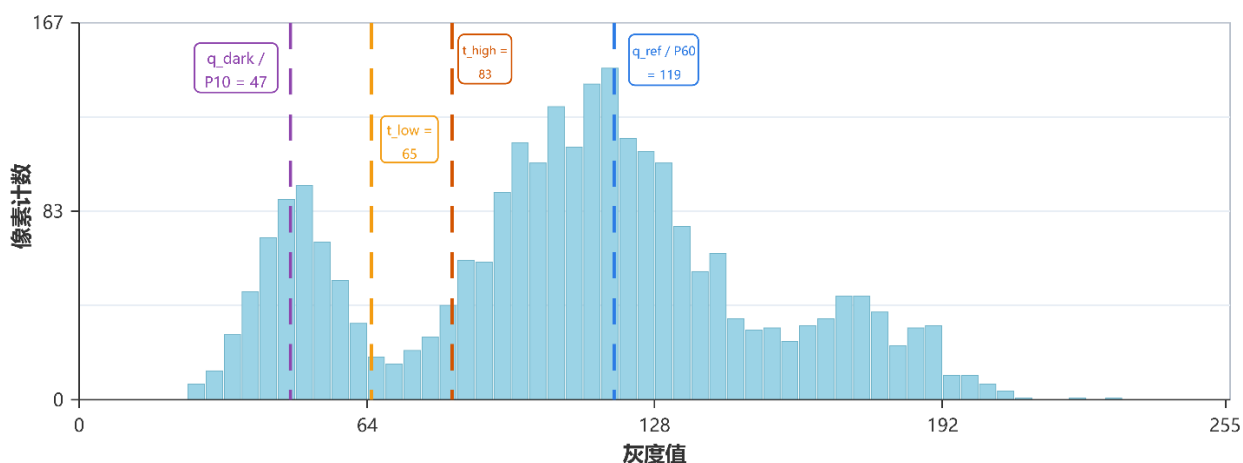
HLS 代码如下：

```
1. int target = div_round_pos(count * percent, 100);
2. for (int i = 0; i < HIST_BINS; i++) {
3.     #pragma HLS PIPELINE II=1
4.     cumulative += hist[i];
5.     if (!found && cumulative >= target) selected = i;
6. }
7. int t = (selected << HIST_SHIFT) + 3;
```

在后续特征统计中，percentile_threshold() 会被调用两次。q_dark 表示 ROI 灰度直方图的低百分位阈值，默认取 10% 分位，用于定位眼部区域中的暗像素范围；q_ref 表示较高百分位参考阈值，默认取 60% 分位，用于估计当前 ROI 的局部亮度水平。二者差值 contrast = q_ref - q_dark 可反映该 ROI 内部的局部对比度。

当 contrast 大于最小对比度要求时，模块进一步由 q_dark 和 q_ref 计算两个实际判定阈值：t_low = q_dark + contrast / 4，t_high = q_dark + contrast / 2。其中 t_low 更严格，主要用于统计较深暗区的连续竖向结构；t_high 更宽松，用于统计眼部暗区比例、top-k 竖向连续长度和有效暗行连续长度。这样可以使阈值随当前 ROI 的灰度分布自适应变化，而不是依赖固定亮度常数。

```
1. int q_dark = percentile_threshold(gray_hist, count, low_pct);
2. int q_ref = percentile_threshold(gray_hist, count, high_pct);
3. if (q_ref < q_dark) q_ref = q_dark;
4.
5. int contrast = q_ref - q_dark;
6. int t_low = q_dark;
7. int t_high = q_dark;
8.
9. if (contrast >= MIN_CONTRAST) {
10.     t_low = q_dark + div_round_pos(contrast, 4);
11.     t_high = q_dark + div_round_pos(contrast, 2);
12. }
```



计算机硬件类综合性课程设计

图 3.1-4 ROI 灰度直方图与 q_dark/q_ref 阈值示意图

(6) 8 维特征定义

本 IP 对每只眼睛先提取四类基础特征，再组合得到 8 个输出特征。四类基础特征分别描述竖向暗区最大连续长度、竖向暗区 top-k 平均长度、暗像素比例和连续有效暗行长度。所有特征均使用 $fixed_scale$ 进行定点归一化，默认 $fixed_scale=4096$ 。

$$f0 = round(max_run_low \times S / h)$$

$$f1 = round(topk_sum_high \times S / (k \times h)), \quad k = [w/4]$$

$$f2 = round(dark_count_high \times S / (w \times h))$$

$$f3 = round(row_run_high \times S / h)$$

表 3.1-2 EyeFeature 单眼基础特征

单眼特征	HLS 变量	物理含义	归一化方式
f0	max_run_low	低阈值下单列最长竖向暗像素连续长度	除以 ROI 高度 h
f1	topk_sum_high	高阈值下 top-k 列竖向暗像素连续长度	除以 $k \times h$
f2	dark_count_high	高阈值下暗像素总数	除以 $w \times h$
f3	row_run_high	高阈值下满足暗像素数量要求的连续行长度	除以 ROI 高度 h

进行双眼融合时，平均特征用于描述两只眼共同的闭眼趋势，差分特征用于保留左右眼不一致信息。若两眼均有效，则输出平均值和绝对差；若只有一眼有效，则平均值退化为该眼特征，差分项置 0。

$$avg_i = (L_i + R_i + 1) \gg 1, \quad diff_i = |L_i - R_i|$$

表 3.1-3 EyeFeature 输出给 SVM 的 8 维特征

输出 index	输出名称	来源
0	avg_f0_q	左右眼 f0 平均值
1	avg_f1_q	左右眼 f1 平均值
2	avg_dark_high_q	左右眼 f2 平均值
3	avg_row_run_high_q	左右眼 f3 平均值
4	diff_f0_q	左右眼 f0 差值
5	diff_f1_q	左右眼 f1 差值
6	diff_dark_high_q	左右眼 f2 差值

(7) 电路设计：硬件数据通路与 HLS 实现

EyeFeature 的硬件数据通路分为初始化、整帧扫描、左右眼统计、双眼融合和输出五个阶段。该设计没有缓存整帧图像，而是在扫描图像流时只保存左右眼内部 ROI 的灰度像素和直方图，因此 BRAM 主要来自 pix0/pix1 两个 ROI 缓冲。

HLS pragma 的使用主要围绕三个目标：接口封装、片上存储推断和循环并行。顶层端口通过 INTERFACE pragma 封装为 AXI-Stream 和 AXI-Lite；pix0/pix1 使用 BIND_STORAGE 指定为双端口 BRAM；直方图和 run 数组使用 cyclic factor=4 的 ARRAY_PARTITION，以降低统计循环中的端口冲突。

1. #pragma HLS INTERFACE axis port=in_stream register_mode=both
2. #pragma HLS INTERFACE axis port=out_stream register_mode=both
3. #pragma HLS INTERFACE s_axilite port=frame_width bundle=CTRL
4. #pragma HLS BIND_STORAGE variable=pix0 type=ram_2p impl=bram
5. #pragma HLS ARRAY_PARTITION variable=hist0 cyclic factor=4 dim=1
6. #pragma HLS PIPELINE II=1

整帧扫描阶段的 FRAME_X 循环设置 PIPELINE II=1，使得在稳定情况下每个时钟周期处理一个输入像素。compute_robust_stats() 不内联，左右眼各形成一个统计实例，便于 HLS 进行模块化调度。

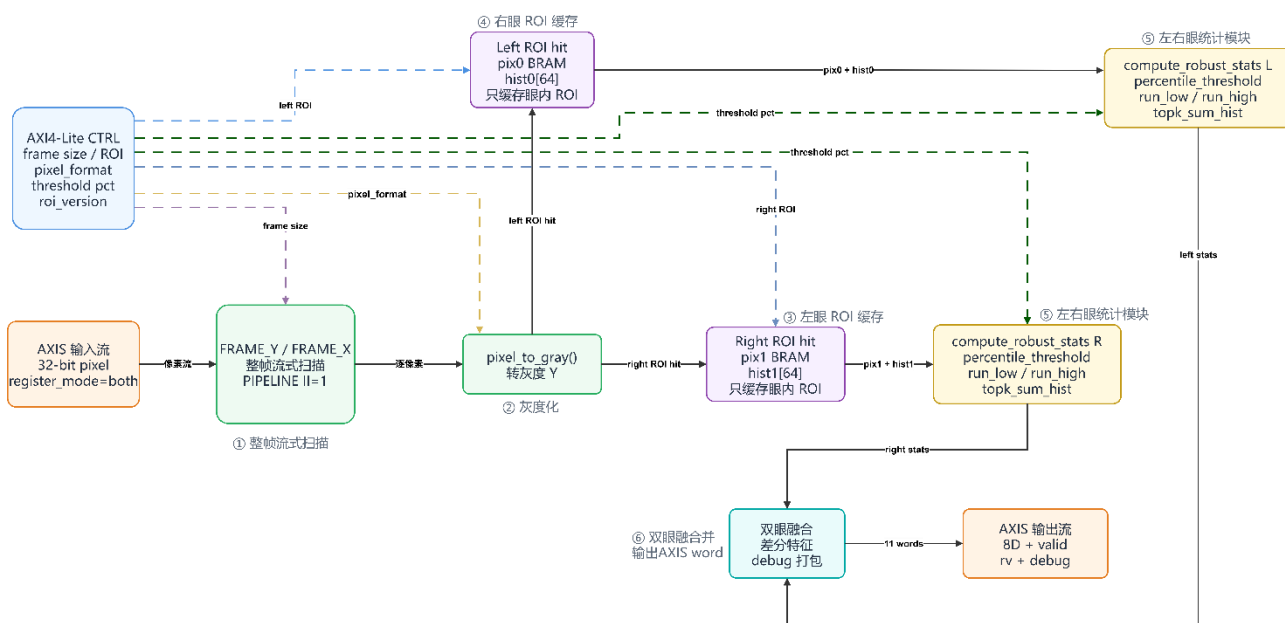


图 3.1-5 EyeFeature HLS 内部模块层次图

(8) IP 封装与寄存器定义

EyeFeature 顶层函数保持 eye_feature 名称，端口由 Vitis HLS 自动封装为一个 AXI-Lite 控制接口和两个 AXI-Stream 数据接口。控制寄存器用于配置当前帧尺寸、左右眼 ROI、阈值参数、输出缩放、像素格式和 ROI 版本号。

计算机硬件类综合性课程设计

```
1. void eye_feature(  
2.     hls::stream<axis_index_t> &in_stream,  
3.     hls::stream<axis_index_t> &out_stream,  
4.     int frame_width, int frame_height,  
5.     int left_x, int left_y, int left_w, int left_h,  
6.     int right_x, int right_y, int right_w, int right_h,  
7.     int roi_valid, int fixed_thresh, int adapt_offset,  
8.     int fixed_scale, int pixel_format, int roi_version);
```

表 3.1-4 EyeFeature AXI-Lite 控制寄存器

寄存器偏移	参数	方向	作用
0x10	frame_width	PS 写	当前输入帧宽度
0x18	frame_height	PS 写	当前输入帧高度
0x20~0x38	left_x/y/w/h	PS 写	左眼 ROI 坐标和尺寸
0x40~0x58	right_x/y/w/h	PS 写	右眼 ROI 坐标和尺寸
0x60	roi_valid	PS 写	当前 ROI 是否有效
0x68	fixed_thresh	PS 写	低百分位阈值参数，默认兼容 10%
0x70	adapt_offset	PS 写	高百分位阈值参数，默认兼容 60%
0x78	fixed_scale	PS 写	输出定点缩放，默认 4096
0x80	pixel_format	PS 写	输入像素格式选择
0x88	roi_version	PS 写	ROI 版本号，随 PS 更新递增

表 3.1-5 EyeFeature AXI-Stream 输出信息

输出 index	名称	说明
0~7	8 维 EyeFeature	按 avg_f0、avg_f1、avg_dark_high、avg_row_run_high、diff_f0、diff_f1、diff_dark_high、diff_row_run_high 顺序输出
8	valid	至少一只眼 ROI 有效且统计成功时为 1
9	roi_version	透传 PS 写入的 ROI 版本号，用于调试 dlib 更新和特征延迟
10	debug_thresholds	打包左右眼 low/high 阈值，TLAST 在该 index 上拉高

对于封装好的 HLS ip，其在 Vivado 的 Block Design 中，暴露输入、输出接口如下图。

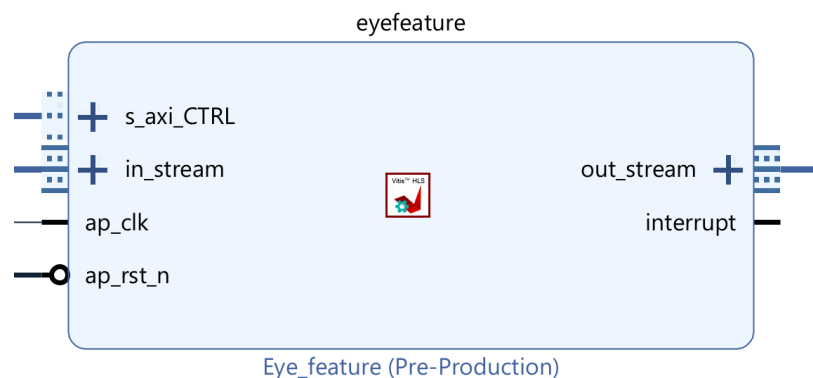


图 3.1-6 EyeFeature 封装后 IP 符号与接口

(9) 时钟、资源与顶层资源占用分析

EyeFeature 使用 Vitis HLS 2023.2 进行综合，目标器件为 xc7z020-clg400-1，时钟约束为 20 ns。HLS 综合报告显示 estimated clock 为 14.458 ns，满足系统 50 MHz 时钟要求。最大延迟为 834004 cycles，约 16.680 ms；延迟主要来自整帧像素扫描，左右眼统计模块各约 9313 cycles。

表 3.1-6 EyeFeature HLS 综合结果

项	HLS 单模块结果
目标周期	20.00 ns
Estimated clock	14.458 ns
最大延迟	834004 cycles / 16.680 ms
FRAME_X	II=1，像素扫描阶段每周期读取一个像素
compute_robust_stats	左右眼各一个实例，单实例最大 9313 cycles
BRAM_18K	8
DSP	57
FF	11386
LUT	25057

需要说明的是，HLS 单模块资源估算不是顶层 bitstream 的最终资源占用。顶层 Vivado post-implementation 后，整套系统约占 LUT 35%、LUTRAM 8%、FF 18%、BRAM 16%、DSP 27%、IO 19%、BUFG 13%。因此报告中将 HLS 结果作为 EyeFeature 模块复杂度参考，将顶层实现结果作为最终 PL 负载依据。

3.1.2 软件设计

(1) PS 端的负载与职责

EyeFeature 的软件部分主要负责为硬件 IP 提供正确的控制参数和数据流。PS 端并不直接计算 8 维 EyeFeature，而是完成 bit/hwh 加载、IP 地址解析、DMA 初始化、ROI 坐标写入、帧数据送入和输出 index 读取。

运行脚本通过 PYNQ Overlay 获取硬件描述，并根据 hwh 中的 IP 字典定位 EyeFeature 实例。由于 HLS IP 在重新封装和替换 block design 后实例名可能变化，软件侧采用按 IP 名称关键词和寄存器特征进行解析的方式，以降低脚本对固定实例名的依赖。

1. 加载 overlay: 读取 bit/hwh, 获取 EyeFeature、EyeFeature DMA 和 SVM DMA 的实例。
2. 初始化寄存器: 写入 frame_width、frame_height、fixed_scale、pixel_format 等固定参数。
3. 更新 ROI: dlib worker 得到新眼部坐标后，写入左右眼 ROI 和 roi_version。
4. 运行 IP: 通过 DMA 将当前帧送入 EyeFeature，读取 11 个输出 index。
5. 递交 SVM: 取 index 0~7 维护 15 帧窗口，并按 feature-major 顺序送入分类 IP。

(2) 寄存器写入顺序与 ROI 版本控制

为了避免硬件运行过程中读到未完全更新的 ROI，软件侧采用相对固定的写入顺序：先写 frame 尺寸和左右眼 ROI 坐标，再写阈值、缩放、像素格式等参数，最后写 roi_valid 和 roi_version。roi_version 随每次 ROI 更新递增，硬件将其原样输出，便于调试时判断某帧特征对应的是哪一次 dlib 检测结果。

```
1. eye.write(FRAME_WIDTH, frame_w)
2. eye.write(FRAME_HEIGHT, frame_h)
3. eye.write(LEFT_X, left.x); eye.write(LEFT_Y, left.y)
4. eye.write(LEFT_W, left.w); eye.write(LEFT_H, left.h)
5. eye.write(RIGHT_X, right.x); eye.write(RIGHT_Y, right.y)
6. eye.write(RIGHT_W, right.w); eye.write(RIGHT_H, right.h)
7. eye.write(FIXED_SCALE, 4096)
8. eye.write(PIXEL_FORMAT, pixel_format)
9. eye.write(ROI_VERSION, roi_version)
10. eye.write(ROI_VALID, 1 if roi_ok else 0)
```

表 3.1-7 EyeFeature 软件寄存器写入项

软件变量	对应寄存器	说明
frame_w / frame_h	frame_width / frame_height	需要与 DMA 送入 EyeFeature 的帧尺寸一致
left/right ROI	left_x...right_h	由 dlib 眼部关键点或眼部框计算得到
roi_ok	roi_valid	无脸、ROI 越界或尺寸异常时置 0
roi_version	roi_version	随 ROI 更新递增，用于调试时确认新旧 ROI
pixel_format	pixel_format	与摄像头链路色彩顺序保持一致

fixed_scale

fixed_scale

与本地训练和 SVM 输入缩放保持一致，默认 4096

(3) 特征读取与窗口构建

EyeFeature 输出 11 个 index，其中 index 0~7 为当前帧的 8 维特征。软件侧读取输出后先检查 valid 和 roi_version，再决定是否将该帧加入窗口。窗口长度为 15 帧，每个特征维护一条时间序列。送入 SVM 时必须按 feature-major、oldest-to-newest 顺序排列。

```
1. svm_input = [
2.     avg_f0[0], ..., avg_f0[14],
3.     avg_f1[0], ..., avg_f1[14],
4.     avg_dark_high[0], ..., avg_dark_high[14],
5.     avg_row_run_high[0], ..., avg_row_run_high[14],
6.     diff_f0[0], ..., diff_row_run_high[14],
7. ]
```

这一顺序是软件训练、HLS SVM IP 和板端运行脚本之间的关键约定。如果顺序发生变化，即使每个单帧特征本身正确，SVM 也会把权重施加到错误的特征位置上，从而导致分类结果失效。

表 3.1-8 由 EyeFeature 到 SVM 的 120D 窗口顺序

窗口区段	发送内容
0~14	avg_f0[0..14]
15~29	avg_f1[0..14]
30~44	avg_dark_high[0..14]
45~59	avg_row_run_high[0..14]
60~74	diff_f0[0..14]
75~89	diff_f1[0..14]
90~104	diff_dark_high[0..14]
105~119	diff_row_run_high[0..14]

(4) 异常处理与内存控制

软件侧对 EyeFeature 输出进行多级检查。若 face=0、roi_valid=0、eye_valid=0 或 DMA 输出长度不满足 11 index，则当前帧不进入 SVM 窗口；若连续较长时间检测不到人脸，则系统进入待机节能状态，降低 Dlib 检测频率并更新 OLED 待机提示。

为了控制 PS 端内存，正常运行脚本不保存视频帧，只保留当前帧、最新 ROI、长度为 15 的特征窗口和疲劳判决滑动窗口。调试模式下才会额外保存带 ROI 框和 pred/score 的视频。这样可以避免长时间运行时视频缓存持续增长。

计算机硬件类综合性课程设计

表 3.1-9 EyeFeature 运行脚本异常处理

异常情况	软件处理方式	目的
无脸或 Dlib 失败	roi_valid=0, 不启动有效窗口更新	避免使用旧 ROI 误触发分类
ROI 越界或尺寸异常	丢弃该次 ROI, 保留或清空有效状态	避免 HLS ROI 缓冲越界
EyeFeature valid=0	当前帧不加入 15 帧窗口	防止无效特征污染 SVM 输入
DMA 输出长度异常	记录错误并跳过分类	保障 AXIS 协议异常时系统不崩溃
长时间无人脸	进入待机节能模式, 低频检测用户返回	降低 PS 端负载和无效提示

3.1.3 模块测试

(1) C Simulation 功能测试

EyeFeature 的 C 仿真主要验证输出 word/index 数量、输出顺序、valid 标志、roi_version 透传和亮度鲁棒性。测试平台构造 96×64 的灰度图像, 在左右眼位置绘制竖向暗线, 分别测试普通亮度、整体变亮和无对比度三类输入。

```

1. std::vector<int> open    = run_case("open",          110, 35, true);
2. std::vector<int> bright  = run_case("open_brighter", 170, 95, true);
3. std::vector<int> flat    = run_case("flat_no_contrast", 180, 180, false);
4. check output size == 11
5. check open[0..7] == bright[0..7]
6. check flat[0..7] == 0
7. check roi_version passthrough == 123

```

表 3.1-10 EyeFeature C 仿真测试用例

测试用例	输入构造	验证目标	结果
open	背景灰度 110, 暗线灰度 35	输出 11 word, valid=1, 存在结构特征	通过
open_brighter	背景灰度 170, 暗线灰度 95	整体亮度变化后特征与 open 一致	通过
flat_no_contrast	背景与线条均为 180	对比度不足时 8 维特征置 0	通过
roi_version	输入版本号 123	输出 word9 保持 123	通过

程序输出结果:

```

[CASE] open words=11 3171 2265 610 3171 0 0 0 0 1 123 ...
[CASE] open_brighter words=11 3171 2265 610 3171 0 0 0 0 1 123 ...
[CASE] flat_no_contrast words=11 0 0 0 0 0 0 0 0 1 123 ...
[PASS] robust8 EyeFeature C test passed

```


(2) HLS 综合与接口测试

综合测试关注三个方面：时钟是否满足 20 ns 约束，接口是否按 AXI-Stream/AXI-Lite 生成，输出 TLAST 是否位于最后一个 debug word。当前 HLS 综合可通过，estimated clock 为 14.458 ns；HW Interfaces 报告中可看到 in_stream、out_stream 和 s_axi_CTRL。

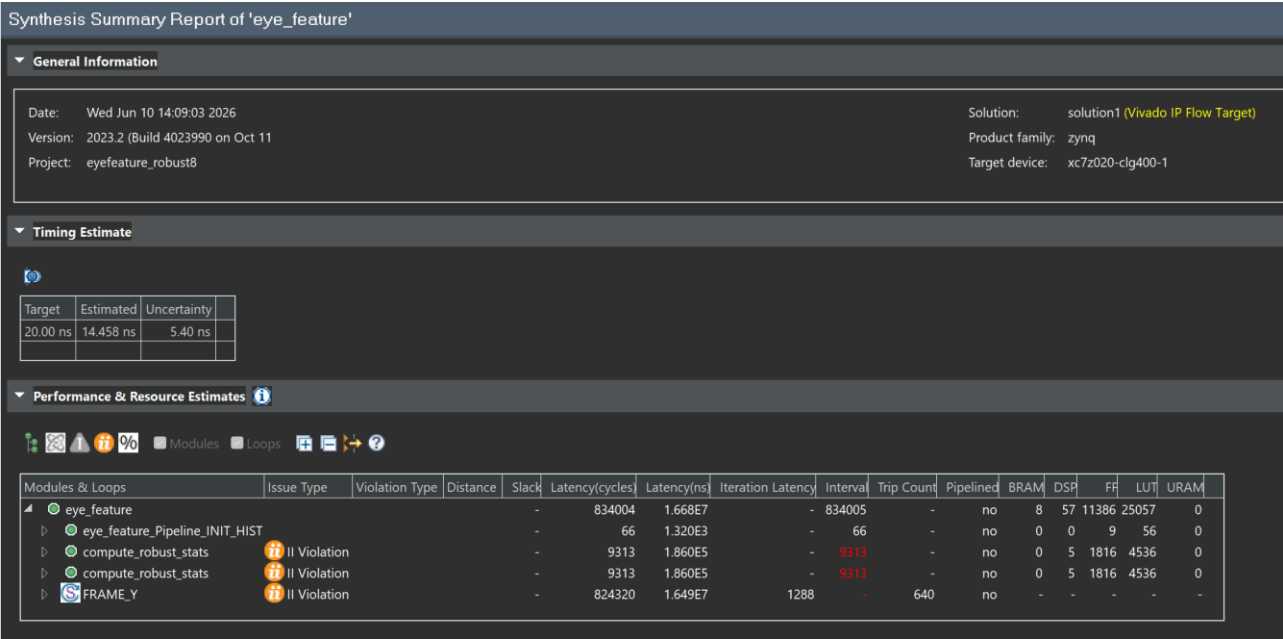


图 3.1-7 EyeFeature HLS 综合接口截图

表 3.1-11 EyeFeature HLS 接口测试检查项

检查项	期望结果	当前状态
时钟约束	Target 20.00 ns，Estimated 小于 target	14.458 ns，通过
输出 word 数	每帧输出 11 个 int32 word	CSIM 通过
TLAST	debug_thresholds 对应 word 拉高 TLAST	代码中 write_out_word(..., true)
AXI-Lite	寄存器覆盖 frame、ROI、阈值、格式和版本号	HLS 接口生成正常
AXI-Stream	输入完整帧，输出特征流	HLS 接口生成正常

(3) 板端单模块测试计划

①测试环境与测试脚本

对 EyeFeature IP 进行独立模块测试。测试在 PYNQ-Z1 上运行，加载 /home/xilinx/LC_SVM/final.bit，对应硬件实例为 eyefeature 和 eyefeature_dma。测试脚本不依赖摄像头和 dlib，而是构造合成灰度帧，通过 AXI DMA 直接送入 EyeFeature IP，从而单独验证该 IP 的接口、寄存器、阈值计算和输出协议。

计算机硬件类综合性课程设计

表 3.1-10 EyeFeature 板端模块测试环境

项目	测试配置
bitstream	/home/xilinx/LC_SVM/final.bit
EyeFeature IP	eyefeature, base=0x43c00000, type=xilinx.com:hls:eye_feature:1.0
EyeFeature DMA	eyefeature_dma, base=0x43c10000, type=xilinx.com:ip:axi_dma:7.1
测试脚本	/home/xilinx/LC_SVM/scripts/test_eyefeature_module.py
测试方式	合成 96×64 灰度帧，左右眼 ROI 固定为 (12,12,32,44) 与 (58,12,32,44)
运行命令	sudo -E bash -lc 'python3 /home/xilinx/LC_SVM/scripts/test_eyefeature_module.py --download --timeout 8'

测试时使用 --download 重新下载 bitstream，目的是在单模块测试前清空 DMA 与 HLS IP 的残留状态，使每次测试从确定的 PL 初始状态开始。

②测试用例设计

合成帧测试包含四个用例。open_gray 和 open_gray_brighter 使用相同的眼部结构，但整体亮度同时抬高；若 EyeFeature 的自适应阈值有效，则两者的结构特征应保持一致。flat_no_contrast 用于验证无局部对比度时不会产生有效结构特征。roi_invalid 用于验证 PS 端写入 roi_valid=0 时，IP 输出 valid=0 且特征清零。

表 3.1-11 EyeFeature 板端测试用例

测试用例	输入构造	验证目标	期望结果
open_gray	背景灰度 110，左右眼暗线灰度 35	正常 ROI 下可输出结构特征	valid=1，8 维特征非零
open_gray_brighter	背景灰度 170，左右眼暗线灰度 95	整体亮度变化后结构特征保持稳定	8 维特征与 open_gray 一致
flat_no_contrast	背景和眼部区域均为灰度 180	局部对比度不足时抑制无意义结构	valid=1，8 维特征全 0
roi_invalid	与 open_gray 相同，但 roi_valid=0	无有效 ROI 时 IP 不输出特征	valid=0，8 维特征全 0

③板端运行结果

实际运行结果如下。open_gray 与 open_gray_brighter 的 8 维特征完全一致，说明该模块在整体亮度平移时主要保留眼部结构信息，而不是直接受绝对灰度值控制。flat_no_contrast 的 8 维特征全部为 0，说明对比度不足时模块不会把均匀灰度误判为眼部结构。roi_invalid 输出 valid=0 且特征全 0，说明 AXI-Lite 的 ROI 有效标志能够正确控制输出。

[EYE_TEST] bit=/home/xilinx/LC_SVM/final.bit download=1

```
[EYE_TEST] resolved eye_ip=eyefeature eye_dma=eyefeature_dma
[EYE_TEST] ip=eyefeature base=0x43c00000 range=0x10000 type=xilinx.com:hls:eye_feature:1.0
[EYE_TEST] ip=eyefeature_dma base=0x43c10000 range=0x10000 type=xilinx.com:ip:axi_dma:7.1
[EYE_TEST] case=open_gray words=11 features=[3171, 2265, 610, 3171, 0, 0, 0, 0] valid=1
roi_version=123 debug=0x49364936 send_dmasr=0x00001002 recv_dmasr=0x00001002
[EYE_TEST] case=open_gray_brighter words=11 features=[3171, 2265, 610, 3171, 0, 0, 0, 0] valid=1
roi_version=124 debug=0x85728572 send_dmasr=0x00001002 recv_dmasr=0x00001002
[EYE_TEST] case=flat_no_contrast words=11 features=[0, 0, 0, 0, 0, 0, 0, 0] valid=1
roi_version=125 debug=0xb7b7b7b7 send_dmasr=0x00001002 recv_dmasr=0x00001002
[EYE_TEST] case=roi_invalid words=11 features=[0, 0, 0, 0, 0, 0, 0, 0] valid=0 roi_version=126
debug=0x00000000 send_dmasr=0x00001002 recv_dmasr=0x00001002
[EYE_TEST] RESULT=PASS
```

表 3.1-12 EyeFeature 板端测试结果

检查项	实际现象	结论
AXI-Stream 输出长度	四个 case 均输出 11 个 word	输出协议正确
亮度鲁棒性	open_gray 与 brighter 的 8 维特征完全一致	自适应阈值工作正常
无对比度抑制	flat_no_contrast 的 8 维特征全 0	对比度保护工作正常
ROI 有效标志	roi_invalid 的 valid=0 且特征全 0	roi_valid 控制正确
ROI 版本透传	输出 roi_version 分别为 123、124、125、126	调试版本号透传正确
DMA 状态	send/recv DMASR 均为 0x00001002	DMA 传输完成且无错误位

④实际情景模拟测试

接下来进行板端控制台 ip 测试。此处使用上节“测试用例设计”所述模拟数据来进行测试。结果如图，均正常工作。（注：此处“index”被命名为“word”）

```
[EYE_TEST] case=open_gray words=11 features=[3171, 2265, 610, 3171, 0, 0, 0, 0] valid=1 roi_version=123 debug=0x49364936 send_dmasr=0x00001002
recv_dmasr=0x00001002
[EYE_TEST] case=open_gray_brighter words=11 features=[3171, 2265, 610, 3171, 0, 0, 0, 0] valid=1 roi_version=124 debug=0x85728572 send_dmasr=0x00001002
recv_dmasr=0x00001002
[EYE_TEST] case=flat_no_contrast words=11 features=[0, 0, 0, 0, 0, 0, 0, 0] valid=1 roi_version=125 debug=0xb7b7b7b7 send_dmasr=0x00001002 recv_dmasr=0x00001002
[EYE_TEST] case=roi_invalid words=11 features=[0, 0, 0, 0, 0, 0, 0, 0] valid=0 roi_version=126 debug=0x00000000 send_dmasr=0x00001002 recv_dmasr=0x00001002
[EYE_TEST] feature_names=avg_f0_q,avg_f1_q,avg_dark_high_q,avg_row_run_high_q,diff_f0_q,diff_f1_q,diff_dark_high_q,diff_row_run_high_q
[EYE_TEST] RESULT=PASS
xilinx@pyng:~$
```

图 3.1-8 EyeFeature 板端模块测试控制台截图

随后进行 EyeFeature 对睁眼/闭眼状态的区分度测试。测试采用板端 OV7670 实际录制的视频数据，分别选取纯睁眼片段和纯闭眼片段中的典型帧，通过 dlib 获取左右眼 ROI，并将同一帧图像送入 EyeFeature IP 进行硬件特征提取。图中绿色框为 dlib 输出的眼部 ROI，青色框为 EyeFeature 实际统计的内部区域。

从图 3.1-7 可以看出，在 ROI 正常覆盖眼部区域的情况下，睁眼与闭眼状态在 EyeFeature 输出上存在可观察差异。以 avg(f0,f1,dark,row) 为例，睁眼帧为 (2273,2398,1408,2561)，闭眼帧为 (2536,2521,1467,2926)。其中 row_run_high 相关特征变化较明显，说明闭眼时眼部暗区在纵向/行方向上的连续结构发生变化。diff 特征也反映出左右

眼局部结构差异，可为后续 SVM 提供补充判别信息。

需要说明的是，单帧特征并不直接作为最终疲劳判断依据。本系统将连续 15 帧 EyeFeature 组成 120 维时间窗口，再输入 SVM IP 进行闭眼分类；最终疲劳状态还会经过 10 s 滑动窗口比例判决。因此，EyeFeature 的作用是提供具有区分度的底层图像结构描述，而不是单独完成疲劳判决。



图 3.1-9 EyeFeature 睁眼/闭眼特征变化分析

此外，我还进行了特征均值对比，用来测试 EyeFeature 的各个 Feature 值对于睁眼/闭眼的区分度和辨别能力。以下图表的结论基于板端 OV7670 录制的视频，有效样本数为：睁眼 1067 帧，闭眼 1046 帧。

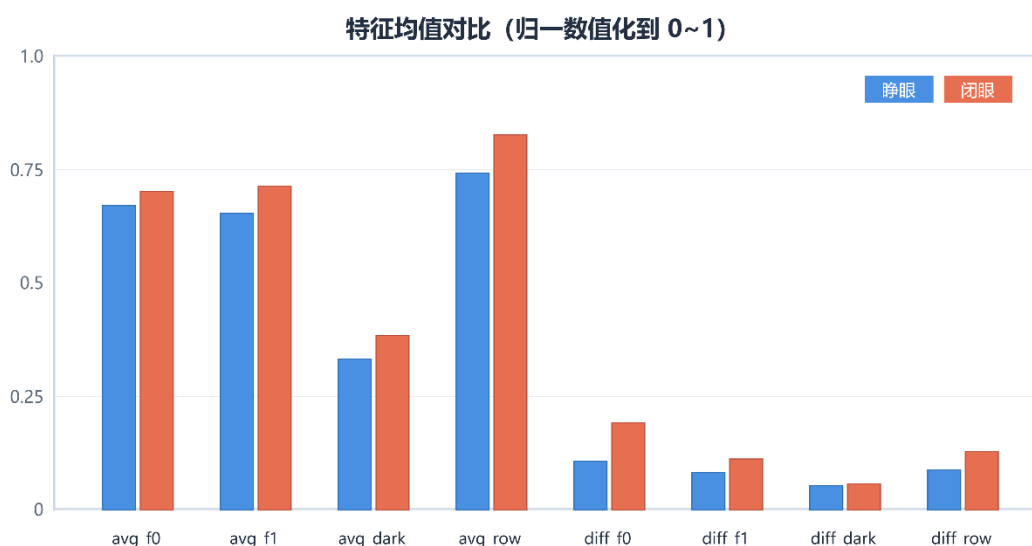


图 3.1-10 EyeFeature 睁眼/闭眼特征均值对比

综上，EyeFeature IP 已完成从 AXI-Lite 配置、AXIDMA 数据传输、输出协议、亮度鲁棒性到异常 ROI 控制的单模块验证，可作为后续 SVM 120D 分类 IP 的稳定输入。

3.2 SVM 120D 分类 IP 的设计（撰写人：侯景祺）

3.2.1 IP 设计

SVM 120D 分类 IP 是 EyeFeature 后级的硬件分类模块，负责把连续 15 帧眼部图像特征转换为当前窗口的闭眼判定。该模块不再执行图像处理，而是对 PS 端整理好的 120 维定点特征向量进行线性 SVM 乘加运算，并输出二分类结果 `pred` 与定点分类分数 `score`。

（1）设计目标与模块功能

该 IP 的设计目标是将 SVM 推理阶段固定为确定时延、低资源占用的 PL 端数据通路。线性 SVM 的模型结构只包含权重向量、偏置和一次内积运算，适合用 HLS 映射为 AXI-Stream 输入、定点乘加、AXI-Stream 输出的硬件模块。

分类对象是 15 帧连续 EyeFeature 组成的窗口。单帧 EyeFeature 为 8 维，窗口长度为 15 帧，因此输入维度为 $8 \times 15 = 120$ 。引入时间窗口后，分类器不仅观察当前帧，还能利用睁眼/闭眼过程中的连续变化，降低单帧 ROI 抖动和瞬时噪声对判定的影响。

输出 `pred` 为二值闭眼结果：0 表示 `open/non_closed`，1 表示 `closed`；`score` 为带符号定点分数，数值越大表示越接近闭眼侧。系统后续的疲劳判决并不是直接使用单帧 `pred`，而是结合 `score` 在滑动时间窗口内统计闭眼比例。

（2）输入输出与窗口顺序

SVM IP 的输入端为 32 bit AXI-Stream，每次运行读取 120 个 `int32` 数据。窗口顺序采用从旧到新的固定协议，即先发送某个特征在 15 帧上的序列，再发送下一个特征。该顺序与本地训练脚本、HLS testbench 和板端 Python 驱动保持一致，是保证权重与输入一一对应的关键。

表 3.2-1 SVM 120D 输入窗口顺序

输入下标范围	特征块	帧内顺序	说明
0~14	<code>avg_f0[0..14]</code>	旧帧 -> 新帧	左右眼 <code>f0</code> 基础特征平均值
15~29	<code>avg_f1[0..14]</code>	旧帧 -> 新帧	左右眼 <code>f1</code> 基础特征平均值
30~44	<code>avg_dark_high[0..14]</code>	旧帧 -> 新帧	左右眼暗像素比例平均值
45~59	<code>avg_row_run_high[0..14]</code>	旧帧 -> 新帧	左右眼横向暗区结构平均值
60~74	<code>diff_f0[0..14]</code>	旧帧 -> 新帧	左右眼 <code>f0</code> 差异
75~89	<code>diff_f1[0..14]</code>	旧帧 -> 新帧	左右眼 <code>f1</code> 差异
90~104	<code>diff_dark_high[0..14]</code>	旧帧 -> 新帧	左右眼暗像素比例差异
105~119	<code>diff_row_run_high[0..14]</code>	旧帧 -> 新帧	左右眼横向暗区结构差异

计算机硬件类综合性课程设计

输出端同样为 32 bit AXI-Stream, 每次运行输出两个 int32 word。第 1 个 word 为 pred, 第 2 个 word 为 score, 且 TLAST 置于第 2 个 word, 方便 DMA 接收端判断本次分类输出结束。

表 3.2-2 SVM IP 外部接口与寄存器

接口/寄存器	方向	位宽/偏移	功能
in_stream	输入	AXI-Stream 32 bit	接收 120 个 int32 定点 EyeFeature 窗口数据
out_stream	输出	AXI-Stream 32 bit	输出 pred 与 score 两个 int32 结果
s_axi_CTRL	输入/输出	AXI-Lite	控制 ap_start/ap_done/ap_idle/ap_ready 与 threshold_q
AP_CTRL	寄存器	0x00	HLS 控制寄存器, bit0 写 1 启动一次分类
threshold_q	寄存器	0x10	分类阈值, score > threshold_q 时输出 pred=1

(3) 线性 SVM 与定点计算原理

线性 SVM 的推理可以写成 $s = w^T x + b$ 。其中 x 为 120 维输入特征, w 为训练后导出的 120 维权重, b 为偏置项。硬件中不再保存浮点数, 而是将输入特征和权重分别放大为整数, 以 64 bit 累加器完成乘加, 最后右移缩放得到 int32 类型的 score。

```

1. s = Σ(w_i · x_i) + b
2. score = round_shift(s, SVM_SCORE_SHIFT)
3. pred = 1, if score > threshold_q; otherwise pred = 0
    
```

表 3.2-3 SVM 定点参数

参数	当前值	说明
SVM_INPUT_DIM	120	输入维度, 即 8 个特征 × 15 帧
SVM_INPUT_SCALE	4096	输入特征定点倍率, $x_q = \text{round}(\text{feature} \times 4096)$
SVM_WEIGHT_SCALE	1048576	训练权重导出后的定点倍率
SVM_SCORE_SHIFT	12	64 bit 累加结果输出前的右移位数
SVM_THRESHOLD_Q	0	默认分类阈值, 对应训练得到的原始判决边界
SVM_B_PRACTICAL	-34489500533	已合入默认阈值的硬件偏置项

threshold_q 是板端可写的运行时阈值, 其单位与 score 相同。它可以理解为在 SVM 输出分数后的可调判决门限, 而不是重新训练模型时改变的权重偏置: threshold_q 越低, 闭眼

越容易触发；threshold_q 越高，闭眼判定越保守。

(4) HLS 顶层函数与核心代码

HLS 顶层函数名为 classify, 与 Vivado Block Design 中的 SVM HLS IP 名称保持一致。in_stream/out_stream 封装为 AXI-Stream, threshold_q 与 return 控制口封装到同一个 AXI-Lite 数据包 (代码中的 bundle)。

```
1. void classify(
2.     hls::stream<axis_t> &in_stream,
3.     hls::stream<axis_t> &out_stream,
4.     data_t threshold_q
5. ) {
6. #pragma HLS INTERFACE axis port=in_stream
7. #pragma HLS INTERFACE axis port=out_stream
8. #pragma HLS INTERFACE s_axilite port=threshold_q bundle=CTRL
9. #pragma HLS INTERFACE s_axilite port=return bundle=CTRL
```

模块内部先用 READ_WINDOW 循环从输入流读入 120 个 int32 数据并暂存到数组 x 中, 然后在 MAC_LOOP 中计算线性 SVM 内积。x 数组采用 cyclic factor=8 的 ARRAY_PARTITION, 使 120 个窗口值被拆分为 8 组小存储, 便于 HLS 调度和读写。

```
1. data_t x[SVM_INPUT_DIM];
2. #pragma HLS ARRAY_PARTITION variable=x cyclic factor=8 dim=1
3. READ_WINDOW:
4. for (int i = 0; i < SVM_INPUT_DIM; ++i) {
5. #pragma HLS PIPELINE II=1
6.     axis_t in_pkt = in_stream.read();
7.     x[i] = (data_t)in_pkt.data;
8. }
9.
10. acc_t acc = SVM_B_PRACTICAL;
11. MAC_LOOP:
12. for (int i = 0; i < SVM_INPUT_DIM; ++i) {
13. #pragma HLS PIPELINE II=1
14.     acc += (acc_t)SVM_W[i] * (acc_t)x[i];
15. }
```

最后将 64 bit 累加值转换为 score, 并与 threshold_q 比较得到 pred。输出函数 write_axis_word() 会写出两个 AXIS word, 并在 score word 上置 TLAST。

```
1. data_t score = score_to_q(acc);
2. data_t pred = (score > threshold_q) ? 1 : 0;
3.
4. write_axis_word(out_stream, pred, false);
5. write_axis_word(out_stream, score, true);
```

(5) IP 封装与寄存器定义

SVMHLS 工程导出后在 Vivado 中作为 classify 类型 IP 使用，在最终 block design 中的实例名为 SVM。该 IP 使用 ap_ctrl_hs 控制协议，PS 端通过 AXI-Lite 写入控制寄存器和 threshold_q；120D 特征输入与 pred/score 输出则通过 svm_dma 的 MM2S/S2MM 两个 AXI4-Stream 通道完成。

在系统地址映射中，SVM IP 的 s_axi_CTRL 端口映射到 0x43c20000，地址范围为 64K；svm_dma 的 AXI-Lite 控制端口映射到 0x43c30000。软件侧通过 hwh 解析实例名后取得 SVM 与 svm_dma 对象，因此后续若 BD 中实例名改变，需要同步检查脚本中的 IP 解析逻辑。

表 3.2-4 SVM IP 封装接口与寄存器定义

接口/寄存器	类型/偏移	软件访问或硬件连接	作用
ap_clk / ap_rst_n	时钟 / 低有效复位	由顶层时钟与复位网络连接，当前 SVM 通路使用 50 MHz	提供 HLS IP 工作时钟与同步复位
s_axi_CTRL	AXI-Lite, base=0x43c20000	PYNQ Overlay 中解析为 SVM 实例	承载 ap_ctrl_hs 控制寄存器和 threshold_q 参数
CTRL	0x00, RW	写 AP_START=1 启动；读 AP_DONE/AP_IDLE/AP_READY 状态	控制一次 SVM 推理的开始与完成
GIE / IP_IER / IP_ISR	0x04 / 0x08 / 0x0C	当前脚本轮询 DMA 与输出结果，未使用中断	HLS 自动生成的中断使能与状态寄存器
threshold_q	0x10, int32, W	svm_ip.write(0x10, threshold_q)	运行时判决门限； pred = score > threshold_q
in_stream	AXI4-Stream, 32 bit	svm_dma MM2S 输出连接到 SVM in_stream	每次接收 120 个 int32 定点特征
out_stream	AXI4-Stream, 32 bit	SVM out_stream 连接到 svm_dma S2MM 输入	输出 2 个 int32： pred 和 score，score word 置 TLAST
interrupt	HLS interrupt 输出	顶层可连接但当前软件未使用	预留中断通知能力

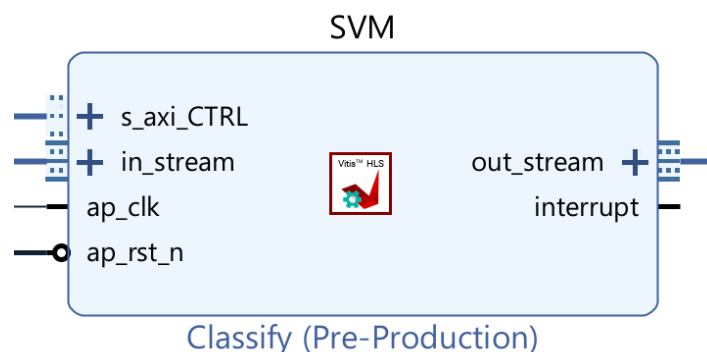


图 3.2-1 SVM IP 在 Vivado Block Design 中的封装图

(6) 时序与资源估计

该 IP 的目标时钟周期设置为 20 ns，对应 50 MHz，与系统中 EyeFeature 和 SVM 通路的时钟配置保持一致。HLS 综合估计时钟为 14.552 ns，满足 20 ns 约束。整体延迟为 253 个周期，即约 5.060 μ s，相比摄像头帧周期和 dlib 处理时间可以忽略。

表 3.2-4 SVM HLS 综合结果

项目	结果	说明
目标器件	xc7z020-clg400-1	PYNQ-Z1 使用的 Zynq-7020
目标时钟	20.00 ns	对应 50 MHz
估计时钟	14.552 ns	满足时序约束
总延迟	253 cycles / 5.060 μ s	一次 120D 窗口分类的完整延迟
READ_WINDOW	122 cycles / 2.440 μ s	读取 120 个 AXIS 输入 word
MAC_LOOP	123 cycles / 2.460 μ s	120 维权重乘加
资源	BRAM_18K=1, DSP=2, FF=544, LUT=1330	LUT 约 2%，资源压力较小

从资源角度看，SVM 120D IP 的硬件压力主要来自 120 次定点乘加和 AXI-Lite 控制逻辑。由于输入维度固定、权重固化在头文件中，模块没有复杂状态机或大容量图像缓存，因此总体资源占用远低于完整图像处理链路。

3.2.2 软件设计

SVM 软件部分包含训练数据构建、权重导出、板端窗口打包、DMA 调用和疲劳判决接口衔接。软件设计的核心要求是：训练阶段、仿真阶段和板端运行阶段必须使用完全一致的 120D 窗口顺序与定点倍率。

(1) 训练数据构建与模型导出

计算机硬件类综合性课程设计

训练数据来自板端 OV7670 摄像头录制的视频，覆盖戴眼镜/不戴眼镜、普通光照/高光照、睁眼/闭眼等场景。视频传回本地后，使用本地 dlib 进行人脸与眼部 ROI 定位，再调用与板端 HLS 行为一致的 EyeFeature 逻辑提取 8 维特征，最后按连续 15 帧组成 120D SVM 样本。

表 3.2-5 SVM 训练数据与配置

项目	当前配置
数据来源	PYNQ-Z1 + OV7670 板端录制视频
场景覆盖	戴眼镜/不戴眼镜，睁眼/闭眼，普通光照/高光照
特征来源	EyeFeature robust8: avg 与 diff 共 8 维
窗口长度	15 帧
样本总数	1959 个窗口样本
类别数量	non_closed/open 983, closed 976
训练/验证/测试	1266 / 291 / 402
模型形式	二分类线性 SVM
权重导出文件	practical_svm_weights_eyefeature_binary.h

模型训练完成后，将浮点权重、偏置和阈值转换为定点整数数组，写入 practical_svm_weights_eyefeature_binary.h。该头文件被 HLS 工程直接 include，因此重新训练后的权重可以通过替换头文件并重新综合 IP 的方式进入 PL。

SVM 训练的混淆矩阵如下图。

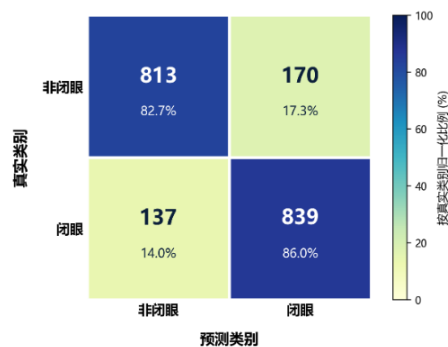


图 3.2-2 SVM 混淆矩阵

(2) 板端 120D 窗口组织

板端运行时，PS 端维护一个长度为 15 的特征缓存队列。每当 EyeFeature 输出有效结果时，将当前帧的 8 维特征加入队列；当队列长度达到 15 后，按 feature-major 顺序展开

为 120 个 int32，写入 SVM DMA 输入缓冲区。

```
1. feature_names = [  
2.     avg_f0, avg_f1, avg_dark_high, avg_row_run_high,  
3.     diff_f0, diff_f1, diff_dark_high, diff_row_run_high  
4. ]  
5. xq = []  
6. for name in feature_names:  
7.     for frame in window_oldest_to_newest:  
8.         xq.append(frame[name])
```

这里的 `window_oldest_to_newest` 是窗口顺序约束中最容易出错的部分。训练脚本、HLS testbench、板端 Python 脚本都必须使用相同顺序，否则同一权重会乘到错误特征上，表现为分类结果整体漂移或完全失效。

(3) AXI DMA 与 AXI-Lite 调用流程

SVM IP 的 120 维输入和 2 维输出均通过 AXI DMA 与 PS DDR 交换。PS 端先将 120 个 int32 写入 send buffer，再准备 2 个 int32 的 recv buffer；随后通过 AXI-Lite 写入 `threshold_q` 和 `ap_start`，等待 DMA 与 IP 完成后读取 `pred/score`。

```
1. # 板端 SVM 调用流程  
2. in_buf[0:120] = xq  
3. out_buf[0:2] = 0  
4. svm_ip.write(0x10, threshold_q)  
5. svm_dma.recvchannel.transfer(out_buf)  
6. svm_dma.sendchannel.transfer(in_buf)  
7. svm_ip.write(0x00, 0x01)  
8. svm_dma.sendchannel.wait(); svm_dma.recvchannel.wait()  
9. pred = int(out_buf[0]); score = int(out_buf[1])
```

当前脚本中 `threshold_q` 可作为运行参数传入，用于修改硬件 `pred` 的判决边界；同时软件疲劳判决还保留 `fatigue-score-threshold-q`，用于统计滑动窗口中的闭眼比例。二者的区别是：`threshold_q` 影响 SVM IP 输出 `pred`，`fatigue-score-threshold-q` 影响最终疲劳窗口统计。

(4) 与疲劳判决逻辑的联系

SVM 的输出是瞬时窗口级闭眼判断，而系统最终报警使用时间窗口统计。板端脚本会持续记录 `score` 与 `pred`，并在 10 s 滑动窗口内统计 `score` 大于疲劳阈值的比例；比例达到设定值后进入警报状态，之后再依据连续睁眼时间和滞回阈值恢复。这样可以避免偶发误判直接触发报警，也能允许闭眼后 SVM 分数存在短暂延迟。

软件还会把 `pred`、`score`、窗口比例、ROI 有效标志、`face/eye/SVM` 状态写入控制台、OLED 显示和运行日志 CSV。这样既便于现场观察，也便于后续重新采集数据、分析阈值和改进训练集。

3.2.3 模块测试

计算机硬件类综合性课程设计

SVM 120D IP 的模块测试分为 HLS C 仿真、综合封装检查、离线数据集验证和板端单模块运行四部分。本次补充的是板端上线后的真实测试结果，测试脚本直接加载当前系统使用的 bitstream，并通过 SVM DMA 向分类 IP 输入 120 维定点向量，从而验证 SVM IP 的数据通路、AXI-Lite 阈值寄存器和 AXIS 输出协议。

(1) 板端测试环境与对象

表 3.2-6 SVM 120D 板端模块测试环境

项目	实测配置
测试板卡	PYNQ-Z1, Zynq-7020, Linux 端通过 sudo -E 运行 PYNQ overlay
bitstream	/home/xilinx/LC_SVM/final_screen.bit
SVM IP	SVM, base=0x43c20000, type=xilinx.com:hls:classify:1.0
SVM DMA	svm_dma, base=0x43c30000, type=xilinx.com:ip:axi_dma:7.1
输入数据	120 个 int32, 顺序为 feature-major、oldest-to-newest
输出数据	2 个 int32, word0=pred, word1=score
AXI-Lite 参数	threshold_q, 偏移地址 0x10, 单位与 score 相同

测试脚本首先加载 overlay 并列出行 hwh 中的 IP 名称，确认 block design 中的 SVM 模块和 DMA 名称能够被软件正确解析。板端关键输出如下：

```
[SVM-MODULE-TEST] begin
[SVM-MODULE-TEST] bit=/home/xilinx/LC_SVM/final_screen.bit
[SVM-MODULE-TEST] expected_dim=120
[SVM-MODULE-TEST] overlay_loaded_sec=2.717
[SVM-MODULE-TEST]
ps=axi_gpio_0,axi_gpio_1,axi_vdma_0,eyefeature,eyefeature_dma,SVM,svm_dma,GPIO_LED,UART_TTS,SPI
SCREEN,processing_system7_0
[SVM-MODULE-TEST] resolved_svm_ip=SVM phys=0x43c20000 dma=svm_dma phys=0x43c30000
```

(2) DMA 输入输出与 120D 数据通路测试

随后测试脚本构造 zero、ramp、pulse 和 packed_feature_major 四组 120D 输入。前三组用于覆盖全零、递增和脉冲型输入，packed_feature_major 则调用板端脚本中的窗口打包函数生成，目的是检查当前软件约定的 15 帧 × 8 特征展开顺序能够被 SVM IP 正确接收。

表 3.2-7 SVM 120D 板端测试向量输出

测试向量	维度	threshold_q=0 时 pred	score	raw 输出	调用耗时/ms
zero	120	0	-8420289	[0, -8420289]	7.402

ramp	120	0	-12014691	[0, -12014691]	4.770
pulse	120	1	11926526	[1, 11926526]	4.817
packed_feature_major	120	0	-1626230	[0, -1626230]	4.749

表中调用耗时为 Python 端一次完整调用测得，包含缓冲区准备、cache flush/invalidate、DMA 启动、等待和读取输出等软件开销，不等同于 HLS 综合报告中的纯硬件分类延迟。HLS 综合结果中的分类延迟约为 253 个周期，在 50 MHz 时钟下约为 5.060 μ s；板端毫秒级耗时主要来自 PS 端驱动和 DMA 调用过程。

(3) threshold_q 寄存器与判决边界测试

为了确认 AXI-Lite 阈值寄存器确实参与硬件判决，测试脚本对每个 case 又进行了两次边界输入：第一次写入 `threshold_q = score - 1`，第二次写入 `threshold_q = score`。实测结果均表现为前者 `pred=1`、后者 `pred=0`，说明 SVM IP 内部采用严格大于关系进行比较。

```
case=zero          score=-8420289  thr_score_minus_1_pred=1  thr_score_pred=0
case=ramp          score=-12014691 thr_score_minus_1_pred=1  thr_score_pred=0
case=pulse         score=11926526  thr_score_minus_1_pred=1  thr_score_pred=0
case=packed_feature_major score=-1626230 thr_score_minus_1_pred=1 thr_score_pred=0
```

由此可得板端 SVM 判决关系为：

```
pred = 1, if score > threshold_q
pred = 0, if score <= threshold_q
```

这一结果也解释了系统调参时 `threshold_q` 的作用：降低 `threshold_q` 会使闭眼判定更容易触发，提高 `threshold_q` 会使判定更保守。`threshold_q` 并不会改变 SVM 的权重或偏置本身，而是在硬件输出 `score` 后提供一个运行时可写的判决门限。

(4) 仿真、综合与离线验证结果

HLS C Simulation 中，测试平台使用相同的定点权重和固定输入向量，软件参考计算值与 HLS 输出一致，验证了 120 个输入 word 的读取数量、score 定点缩放、pred 输出以及 TLAST 位置。

```
[CASE] zero exp=0 got=0 score exp=-8420289 got=-8420289
[CASE] ramp exp=0 got=0 score exp=-6226208 got=-6226208
[CASE] pulse exp=0 got=0 score exp=-8527636 got=-8527636
[PASS] classify csim passed
```

表 3.2-8 SVM 模块测试结论汇总

检查项	实测现象	结论
Overlay/IP 解析	hwh 中解析到 SVM 与 <code>svm_dma</code> ，基地址分别为 0x43c20000 和 0x43c30000	IP 名称与地址映射正确
AXI DMA 输入	四组 case 均成功发送 120 个 int32 输入 word	输入维度与 DMA 通路正确

计算机硬件类综合性课程设计

AXIS 输出协议	每次运行返回 raw=[pred, score] 两个 int32	输出协议正确
threshold_q 寄存器	threshold_q=score-1 时 pred=1; threshold_q=score 时 pred=0	AXI-Lite 阈值寄存器有效
HLS 综合时序	20 ns 目标周期下 estimated clock 约 14.552 ns, 延迟约 253 cycles	50 MHz 约束下满足时序
离线数据集验证	基于板端 OV7670 录制视频构建 robust8+120D 数据集, 离线测试 micro accuracy 约 84.33%	权重来源具有数据集支撑

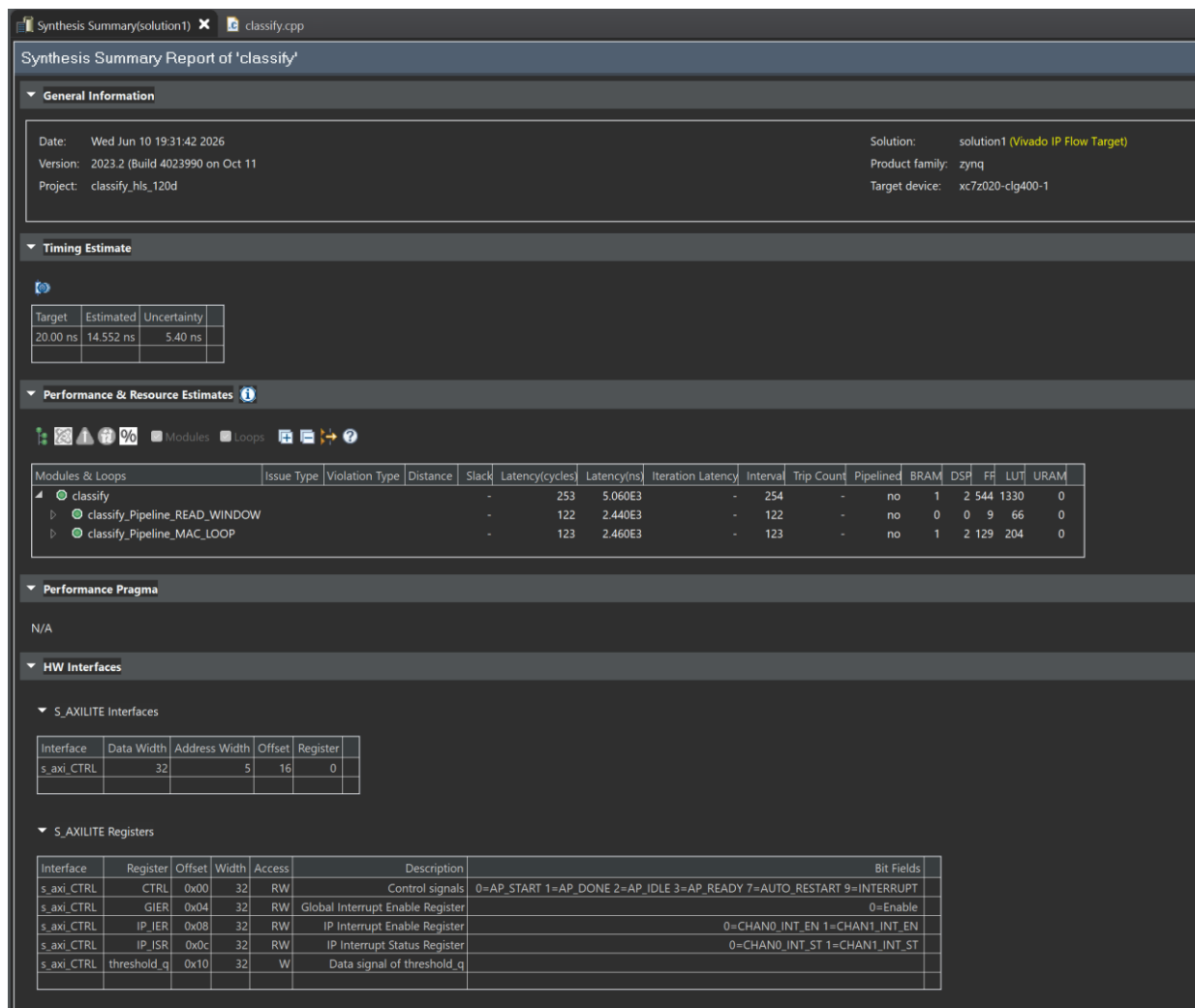


图 3.2-3 SVM HLS Synthesis 图

综上, SVM 120D 分类 IP 已完成从 HLS C 仿真、综合封装到端单模块运行的验证。板端测试证明, SVM IP 能够通过 AXI DMA 正确接收 120D 定点输入并返回 pred/score, 同时 threshold_q 可通过 AXI-Lite 在运行时修改判决边界。后续整机演示中的疲劳报警逻辑基于该 score 输出继续进行 10 s 滑动窗口统计。

3.3 OV7670 IP 模块的设计（撰写人：李佳萌）

3.3.1 IP 设计

（1）OV7670 IP 实现的功能

OV7670 IP 的功能边界是从摄像头物理接口到 AXI4-Stream 视频流接口的转换。它不负责后续算法识别，也不在内部保存完整图像，而是把摄像头初始化、DVP 数据接收、RGB565 像素拼接、视频流封装这几个硬件步骤整理成一个可复用模块。这样在系统集成时，后级只需要接收标准 AXI4-Stream 视频流，再交给 AXI Data FIFO、AXI VDMA 和 DDR 帧缓存即可。

① 摄像头外设控制：系统 clk 进入 IP 后，一方面作为 SCCB 初始化模块的控制时钟，另一方面经过简单分频产生 ov_xclk，提供给 OV7670 作为外部工作时钟。同时模块输出 ov_reset_n 和 ov_pwrn 控制摄像头复位和掉电状态。设计中 ov_pwrn 固定为 0，表示摄像头不进入掉电模式；ov_reset_n 在上电初期先保持低电平，等待约 5 ms 后再拉高，使摄像头完成硬件复位。

② SCCB 寄存器初始化：OV7670 上电后默认输出格式、窗口范围、颜色矩阵等参数不能直接满足本系统要求，因此必须先通过 SCCB 总线写寄存器。IP 内部的 ov7670_reg_rom_rgb565 用 ROM 形式保存初始化表，ov7670_sccb_init 按 index 顺序读出寄存器地址和数据，再交给 sccb_master 产生 SIO_C/SIO_D 时序，实现逐项写入。

③ RGB565 输出配置：寄存器表主要配置摄像头输出 RGB565 格式，同时设置图像窗口、颜色相关参数和测试模式关闭状态。这样 DVP 端每两个 8 位数据可以组成一个 16 位 RGB565 像素，后续软件端也可以按 RGB565 规则从低 16 位还原图像。

④ DVP 数据采集：摄像头输出的 cam_pclk 是数据采样时钟，cam_vsync 表示帧同步，cam_href 表示当前行有效，cam_data[7:0] 是 8 位并行图像数据。DVP 采集模块在 cam_pclk 上升沿采样这些信号，并根据 VSYNC/HREF 的边沿产生 frame_start、line_start、line_end 等标志。

⑤ 像素拼接与坐标计数：由于 RGB565 一个像素占 16 位，而 OV7670 的 DVP 数据总线一次只输出 8 位，因此模块使用 byte_phase 标志记录当前采集的是一个像素的第一个字节还是第二个字节。只有收到第二个字节后才输出 pixel_valid，同时给出 pixel_x 和 pixel_y，供后级判断行尾和有效分辨率。

⑥ AXI4-Stream 视频流输出：to_vdma_axis 模块把 pixel_data[15:0] 封装为 32 位 AXI4-Stream 数据。m_axis_tdata[15:0] 保存真实 RGB565 像素，m_axis_tdata[31:16] 补 0；tuser 标记帧首，tlast 标记行尾，tvalid/tready 完成握手。该接口可以直接连接 AXI Data FIFO 或 AXI VDMA。

⑦ 状态与调试输出：IP 输出 init_busy、init_done、init_error、axis_frame_done、axis_overflow 以及 dbg_index、dbg_reg_addr、dbg_reg_data、dbg_state 等调试信号。调试时可通过 ILA 观察这些信号，判断问题发生在 SCCB 初始化、DVP 采集、AXI 握手还是 VDMA 接收阶段。

（2）OV7670 IP 工作原理

OV7670 IP 的工作可以按照“上电复位—寄存器初始化—DVP 采集—像素拼接—AXI4-Stream 输出”来理解。系统 clk 域主要负责摄像头初始化，cam_pclk 域主要负责视频数据采集和输出。两部分通过 init_done、capture_enable 等控制信号衔接，保证摄像头配置完成后才开始传输有效图像。

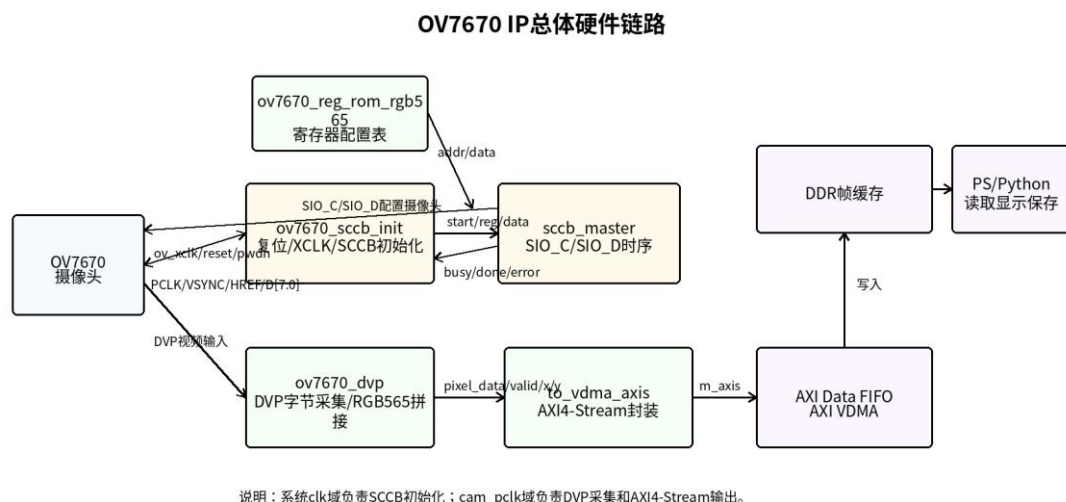


图 3.3-1 OV7670 IP 总体硬件链路

① 摄像头接口与时钟关系

摄像头侧一共包含控制接口、SCCB 配置接口和 DVP 视频接口三类信号。控制接口包括 `ov_xclk`、`ov_reset_n` 和 `ov_pwdn`；SCCB 配置接口包括 `sccb_sio_c` 和 `sccb_sio_d`；DVP 视频接口包括 `cam_pclk`、`cam_vsync`、`cam_href` 和 `cam_data[7:0]`。其中 `ov_xclk` 由 FPGA 输出给摄像头，`cam_pclk` 由摄像头返回给 FPGA。也就是说，寄存器配置过程受系统 `clk` 控制，而图像字节采集必须跟随 `cam_pclk` 进行。

在 `ov7670_sccb_init` 中，`ov_xclk` 由 2 位计数器 `xclk_div` 分频得到，代码中 `assign ov_xclk = xclk_div[1]`。当系统时钟为 100 MHz 时，该信号约为 25 MHz，接近 OV7670 常用的外部时钟要求。虽然目标时钟通常写作 24 MHz，但实际工程中可以由 Clocking Wizard 或分频逻辑提供相近频率，关键是摄像头能够稳定输出 `PCLK`、`VSNC`、`HREF` 和数据。

```
1. reg [1:0] xclk_div;
2.
3. always @(posedge clk or negedge rst_n) begin
4.     if (!rst_n)
5.         xclk_div <= 2'd0;
6.     else
7.         xclk_div <= xclk_div + 1'b1;
8. end
9.
10. assign ov_xclk = xclk_div[1];
11. assign ov_pwdn = 1'b0;
```

这段代码体现了摄像头外部控制的基本思路：`ov_pwdn` 固定为 0，保证摄像头始终处于非掉电状态；`ov_xclk` 通过计数器分频产生，使摄像头获得连续工作时钟；复位信号 `ov_reset_n` 则由后面的初始化状态机控制。

② SCCB 初始化流程

SCCB 是 OV7670 用于寄存器配置的串行控制总线，与 I2C 的使用方式很接近。初始化时，主控逻辑先等待摄像头硬件复位稳定，然后向寄存器 0x12 写入 0x80 执行软件复位。软件复位结束后，状态机从 ROM 的 `index=0` 开始逐项读取寄存器地址和数据，通过 `sccb_master` 写入 OV7670。写到最后一项时，ROM 输出 `last=1`，状态机进入 `S_DONE`，

init_done 拉高。

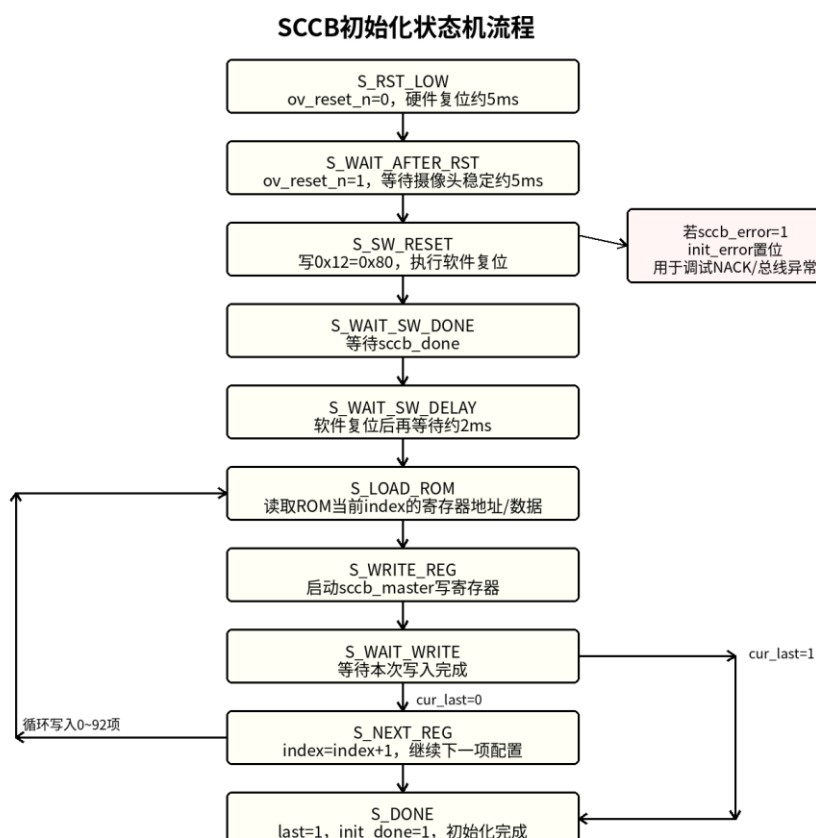


图 3.3-2 SCCB 初始化状态流程图

初始化状态机的状态划分比较清晰：S_RST_LOW 和 S_WAIT_AFTER_RST 负责硬件复位前后的延时；S_SW_RESET、S_WAIT_SW_DONE 和 S_WAIT_SW_DELAY 负责软件复位；S_LOAD_ROM、S_WRITE_REG、S_WAIT_WRITE 和 S_NEXT_REG 负责循环写入寄存器表；S_DONE 表示所有寄存器配置完成。关键代码如下：

```

1. localparam integer WAIT_5MS = CLK_FREQ_HZ / 200;
2. localparam integer WAIT_2MS = CLK_FREQ_HZ / 500;
3.
4. localparam [3:0]
5.     S_RST_LOW      = 4'd0,
6.     S_WAIT_AFTER_RST = 4'd1,
7.     S_SW_RESET     = 4'd2,
8.     S_WAIT_SW_DONE  = 4'd3,
9.     S_WAIT_SW_DELAY = 4'd4,
10.    S_LOAD_ROM      = 4'd5,
11.    S_WRITE_REG     = 4'd6,
12.    S_WAIT_WRITE    = 4'd7,
13.    S_NEXT_REG      = 4'd8,
14.    S_DONE          = 4'd9;
    
```

其中 WAIT_5MS 和 WAIT_2MS 都根据 CLK_FREQ_HZ 自动计算，因此当系统时钟频率改变时，只需要修改参数即可，不需要手动重新计算等待计数值。这样的写法比固定计数常量更利于 IP 复用。

```

1. S_SW_RESET: begin
2.   if (!sccb_busy) begin
3.     sccb_rw      <= 1'b0;
4.     sccb_reg_addr <= 8'h12;
5.     sccb_wr_data <= 8'h80;
6.     sccb_start   <= 1'b1;
7.     state        <= S_WAIT_SW_DONE;
8.   end
9. end
10.
11. S_LOAD_ROM: begin
12.   dbg_index      <= rom_index;
13.   dbg_reg_addr   <= rom_reg_addr;
14.   dbg_reg_data   <= rom_reg_data;
15.
16.   if (!rom_valid) begin
17.     state <= S_DONE;
18.   end else begin
19.     cur_reg_addr <= rom_reg_addr;
20.     cur_reg_data <= rom_reg_data;
21.     cur_last    <= rom_last;
22.     state       <= S_WRITE_REG;
23.   end
24. end

```

这段代码中，写 0x12=0x80 是 OV7670 的软件复位步骤。S_LOAD_ROM 状态会把当前 ROM 输出锁存到 cur_reg_addr 和 cur_reg_data 中，避免状态跳转过程中 ROM 数据变化影响 SCCB 写入。dbg_index、dbg_reg_addr 和 dbg_reg_data 同步输出，便于用 ILA 观察当前写到第几项、写入的是哪个寄存器。

③ SCCB 主机时序设计

sccb_master 负责把上层传入的 reg_addr 和 wr_data 转换成 SIO_C/SIO_D 上的实际时序。单次写寄存器的顺序为：START 起始条件、OV7670 写地址 0x42、ACK、寄存器地址、ACK、寄存器数据、ACK、STOP 停止条件。读操作也预留了逻辑，读地址为 0x43。

SCCB单寄存器写入时序示意



SCL：每个字节8个时钟，ACK为第9个时钟

SDA：SCL为高电平时保持稳定；START为SCL高时SDA由高到低，STOP为SCL高时SDA由低到高

本设计写寄存器流程：start=1 → busy=1 → 依次发送地址/寄存器/数据 → done拉高1个clk。

图 3.3-3 SCCB 单寄存器写入时序示意图

由于 SIO_D 是双向数据线，发送数据时 FPGA 要驱动它，接收 ACK 或读数据时

FPGA 要释放它。代码中使用 IOBUF 实现三态控制, sda_flag 为 1 时表示 FPGA 主动驱动 SDA, sda_flag 为 0 时表示释放 SDA, 让外部上拉或摄像头驱动。

```
1. localparam [7:0] OV7670_WR_ID = 8'h42;
2. localparam [7:0] OV7670_RD_ID = 8'h43;
3.
4. IOBUF IOBUF_inst (
5.     .O (sda_in),
6.     .IO (sio_d),
7.     .I (sda_reg),
8.     .T (~sda_flag)
9. );
```

SCCB 时钟由系统 clk 分频产生。代码中把一个 SCCB 位周期划分为 4 个 tick, 用 ST_TX_SETUP、ST_TX_HIGH1、ST_TX_HIGH2、ST_TX_LOW 等状态控制数据建立、SCL 拉高保持和 SCL 拉低。这样可以保证 SDA 在 SCL 高电平期间保持稳定, 符合 SCCB/I2C 类总线“高电平采样、低电平变化”的基本要求。

```
1. localparam integer RAW_TICK_DIV = CLK_FREQ_HZ / (SCCB_FREQ_HZ * 4);
2. localparam integer TICK_DIV      = (RAW_TICK_DIV < 1) ? 1 : RAW_TICK_DIV;
3.
4. ST_TX_SETUP: begin
5.     sio_c <= 1'b0;
6.     if (tx_shift[bit_cnt] == 1'b0) begin
7.         sda_reg <= 1'b0;
8.         sda_flag <= 1'b1; // 发送 0: 主动拉低 SDA
9.     end else begin
10.        sda_reg <= 1'b0;
11.        sda_flag <= 1'b0; // 发送 1: 释放 SDA
12.    end
13.    state <= ST_TX_HIGH1;
14. end
```

在发送逻辑中, 发送 0 时由 FPGA 主动拉低 SIO_D; 发送 1 时释放 SIO_D, 由上拉电阻形成高电平。这种写法符合开漏/准双向总线的使用方式, 可以避免 FPGA 与摄像头同时驱动数据线造成冲突。

④ 寄存器 ROM 配置

ov7670_reg_rom_rgb565 是一个组合逻辑 ROM, 通过 index 输入输出对应的 reg_addr、reg_data、valid 和 last。当前 ROM 中 index=0 到 index=92 为有效配置, 共 93 项; index=92 时 last=1, 表示寄存器表最后一项。若 index 超出范围, valid=0, 避免状态机继续写入无效寄存器。

```
1. 8'd92: begin
2.     reg_addr = 8'h42;
3.     reg_data = 8'h00;
4.     last     = 1'b1;
5. end
6.
7. default: begin
```

```

8.   reg_addr = 8'hFF;
9.   reg_data = 8'hFF;
10.  valid    = 1'b0;
11.  last     = 1'b0;
12. end

```

寄存器表中的配置项可以按作用分为几类：0x11 用于时钟相关设置；0x12、0x40、0x3A 等与输出格式和 RGB565 数据顺序有关；0x17、0x18、0x19、0x1A、0x32、0x03 等与图像窗口范围有关；0x4F 到 0x54、0x58 等与颜色矩阵和色彩校正有关；0x70、0x71、0x42 等与缩放、测试图案或相关输出控制有关。由于摄像头图像质量和颜色正确性高度依赖这些寄存器，调试时如果出现颜色不对、图像偏移或测试彩条异常，首先应检查 ROM 表中的这些配置。

⑤ DVP 帧同步、行同步与字节采集

OV7670 的 DVP 数据不是一次输出一个完整像素，而是在 `cam_pclk` 的每个有效周期输出一个 8 位字节。VSYNC 用于表示一帧的同步状态，HREF 用于表示当前行是否有效。代码中先对 `cam_vsync`、`cam_href` 和 `cam_data` 进行一级寄存，再通过当前值和前一拍值判断边沿，避免直接使用外部输入造成组合毛刺。

```

1. vsync_d <= vsync_q;
2. href_d  <= href_q;
3.
4. vsync_q <= cam_vsync;
5. href_q  <= cam_href;
6. data_q  <= cam_data;
7.
8. assign vsync_active_fall = vsync_active_pre && (!vsync_active_now);
9. assign href_active_rise  = (!href_active_pre) && href_active_now;
10. assign href_active_fall = href_active_pre && (!href_active_now);

```

本设计将 VSYNC 高电平视为帧消隐/帧同步有效，VSYNC 从有效变为无效时，表示新一帧有效图像即将开始，因此在 `vsync_active_fall` 时产生 `frame_start`，并清零 `x_cnt`、`y_cnt` 和 `byte_phase`。HREF 上升沿表示新一行开始，此时 `x_cnt` 清零；HREF 下降沿表示当前行结束，如果该行已经接收过像素，则 `y_cnt` 加 1。

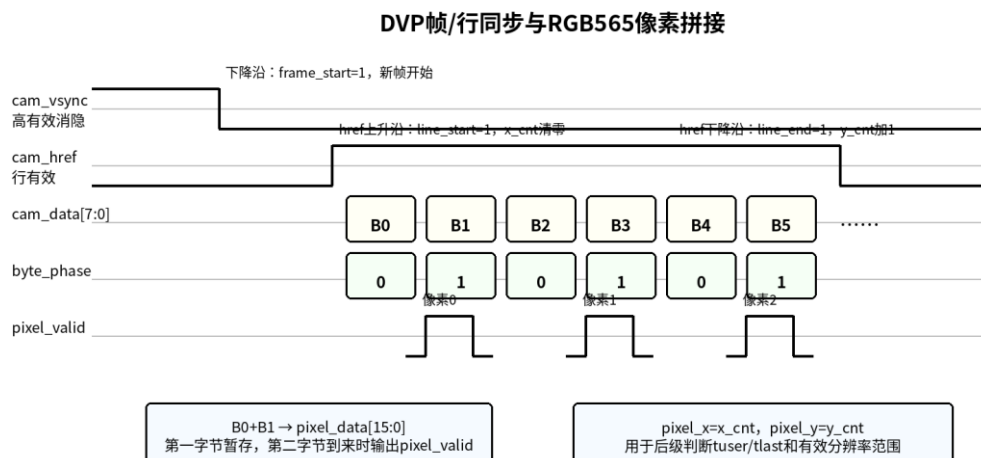


图 3.3-4 DVP 帧/行同步与 RGB565 像素拼接关系

采集到有效字节后，模块通过 `byte_phase` 完成两个字节到一个像素的拼接。当 `byte_phase=0` 时，当前字节只是一个像素的第一个字节，先保存到 `first_byte`；当 `byte_phase=1` 时，当前字节是第二个字节，此时与 `first_byte` 组合成 `pixel_data`，并产生一个周期的 `pixel_valid`。

```

1. if (current_byte_valid) begin
2.     dbg_byte_data <= sample_byte;
3.     dbg_byte_valid <= 1'b1;
4.
5.     if (byte_phase == 1'b0) begin
6.         first_byte <= sample_byte;
7.         byte_phase <= 1'b1;
8.         dbg_byte_phase <= 1'b1;
9.     end else begin
10.        if (SWAP_BYTES)
11.            pixel_data <= {sample_byte, first_byte};
12.        else
13.            pixel_data <= {first_byte, sample_byte};
14.
15.        pixel_valid <= 1'b1;
16.        pixel_x <= x_cnt;
17.        pixel_y <= y_cnt;
18.        x_cnt <= x_cnt + 1'b1;
19.        line_has_pixel <= 1'b1;
20.        byte_phase <= 1'b0;
21.    end
22. end

```

这里设置 `SWAP_BYTES` 参数是为了给调试预留字节交换能力。如果摄像头输出字节顺序与后级期望不一致，可以通过该参数调整拼接顺序。`pixel_x` 和 `pixel_y` 在 `pixel_valid` 同时输出，保证每一个有效像素都有对应坐标。后级 AXI 模块正是根据这些坐标判断该像素是否在 640×480 有效范围内，以及该像素是否为行尾或帧首。

⑥ AXI4-Stream 视频流输出

`to_vdma_axis` 模块的任务是把 DVP 模块输出的像素流转换为 VDMA 能够接收的 AXI4-Stream 视频流。模块首先对 `capture_enable` 做两级寄存同步，避免外部使能信号直接进入 `cam_pclk` 域造成亚稳态。只有 `cap_en` 有效时，AXI 输出逻辑才工作；如果 `cap_en` 无效，`tvalid`、`tlast`、`tuser` 和计数器全部清零。

```

1. reg cap_en_d1;
2. reg cap_en_d2;
3.
4. always @(posedge pclk or negedge rst_n) begin
5.     if (!rst_n) begin
6.         cap_en_d1 <= 1'b0;
7.         cap_en_d2 <= 1'b0;
8.     end else begin
9.         cap_en_d1 <= capture_enable;

```

```

10.      cap_en_d2 <= cap_en_d1;
11.  end
12. end
13.
14. wire cap_en = cap_en_d2;

```

AXI4-Stream 协议中，只有 tvalid 和 tready 同时为 1 时，一个数据才算真正传输成功。代码中 axis_fire 表示一次有效握手，can_load_new 表示当前输出寄存器可以装载新的像素。如果当前 tvalid 已经为 1 但 tready 为 0，说明下游暂时不能接收数据，此时必须保持当前 tdata 不变，不能覆盖。

```

1. wire axis_fire;
2. assign axis_fire = m_axis_tvalid && m_axis_tready;
3.
4. wire can_load_new;
5. assign can_load_new = (!m_axis_tvalid) || axis_fire;
6.
7. wire pixel_in_range;
8. assign pixel_in_range =
9.     axis_frame_active &&
10.    pixel_valid &&
11.    (pixel_x < FRAME_WIDTH) &&
12.    (pixel_y < FRAME_HEIGHT);

```

pixel_in_range 用于过滤有效分辨率范围之外的像素。这样做的原因是摄像头输出的一帧中可能包含消隐区或多余像素，而 VDMA 期望接收固定宽度和高度的有效图像。FRAME_WIDTH=640、FRAME_HEIGHT=480 时，模块只将坐标小于该范围的像素送入后级。

RGB565像素到AXI4-Stream信号映射

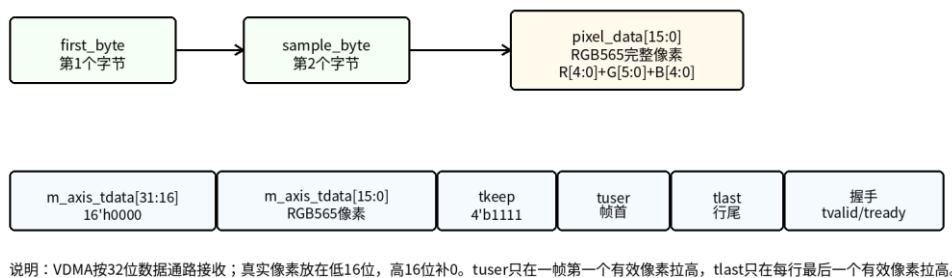


图 3.3-5 RGB565 像素到 AXI4-Stream 信号的对应关系

```

1. assign m_axis_tkeep = 4'b1111;
2.
3. if (can_load_new && pixel_in_range) begin
4.     m_axis_tdata <= {16'h0000, pixel_data};
5.     m_axis_tvalid <= 1'b1;
6.     m_axis_tuser <= is_first_pixel;
7.     m_axis_tlast <= is_line_last;
8.

```



```

9.    axis_sent_pixel_count <= axis_sent_pixel_count + 1'b1;
10.
11.    if (is_frame_last) begin
12.        axis_frame_done    <= 1'b1;
13.        axis_frame_active <= 1'b0;
14.    end
15. end

```

m_axis_tdata 的低 16 位保存 RGB565 像素，高 16 位补 0，是为了和 32 位 AXI 数据通路对齐。tkeep 固定为 4'b1111，表示 4 个字节都参与传输；tuser 在 pixel_x=0 且 pixel_y=0 时拉高，用于标记一帧开始；tlast 在 pixel_x=FRAME_WIDTH-1 时拉高，用于标记一行结束；axis_frame_done 在最后一个有效像素发送时拉高一个周期。

如果 m_axis_tvalid 已经拉高但下游 m_axis_tready 没有响应，同时新的 pixel_valid 又到来，就说明摄像头输入速度超过了后级接收能力，可能引起丢像素、错位或撕裂。因此代码中设置 axis_overflow 作为背压报警信号，便于调试 AXI Data FIFO、VDMA 和帧缓存配置。

```

1. if (m_axis_tvalid && !m_axis_tready && pixel_valid) begin
2.    axis_overflow <= 1'b1;
3. end

```

(3) 电路设计

从电路结构上看，顶层 ov7670_camera_top 主要起到“模块组织”和“接口整理”的作用，真正的功能分别放在 ov7670_sccb_init、sccb_master、ov7670_reg_rom_rgb565、ov7670_dvp 和 to_vdma_axis 中。这样的分层方式便于独立调试：先看 SCCB 是否完成寄存器写入，再看 DVP 是否输出连续 pixel_valid，最后看 AXI4-Stream 是否能够被 VDMA 正常接收。

表 3.3-1 OV7670 IP 内部子模块划分

子模块	主要功能	关键输入/输出
ov7670_sccb_init	产生 ov_xclk，控制 ov_reset_n/ov_pwrn，按照寄存器 ROM 启动 SCCB 初始化流程，并输出 init_busy/init_done/init_error。	clk、rst_n、ov_xclk、ov_reset_n、sccb_sio_c、sccb_sio_d、init_done、dbg_state
sccb_master	实现 SCCB 单字节寄存器读写时序，包括 START、地址发送、寄存器地址、数据发送、ACK/NA 检测和 STOP。	start、rw、reg_addr、wr_data、busy、done、error、sio_c、sio_d
ov7670_reg_rom_rgb565	保存 RGB565 模式下的 OV7670 初始化寄存器表，通过 index 输出当前配置项，并	index、reg_addr、reg_data、valid、last

在最后一项给出 last。

ov7670_dvp

在 cam_pclk 域采集 VSYNC/HREF/D[7:0], 两个字节拼接为一个 RGB565 像素, 同时产生坐标和行帧标志。

cam_pclk 、 cam_vsync 、
cam_href 、 cam_data 、
pixel_data 、 pixel_valid 、
pixel_x、 pixel_y

to_vdma_axis

把像素流封装为 AXI4-Stream, 产生 tdata、tkeep、tvalid、tready、tlast、tuser, 并统计帧完成和溢出状态。

capture_enable、pixel_data、
pixel_valid 、 m_axis_* 、
axis_frame_done 、
axis_overflow

① ov7670_sccb_init 子模块设计

该模块是摄像头初始化控制核心。它内部同时完成三件事：第一，产生 ov_xclk 和摄像头复位/掉电控制信号；第二，从 ov7670_reg_rom_rgb565 中读取寄存器配置；第三，驱动 sccb_master 执行写寄存器操作。模块提供 init_busy、init_done 和 init_error 三个状态输出，便于软件端或 ILA 判断初始化阶段是否正常。

该模块的状态机不是一次性把所有寄存器同时送出，而是采用“读 ROM—锁存当前寄存器—启动 SCCB—等待 done—判断 last—进入下一项”的顺序执行。这样可以保证每一项 SCCB 写操作完成后进入下一项，避免总线冲突。

```

1. S_WRITE_REG: begin
2.   if (!sccb_busy) begin
3.     sccb_rw      <= 1'b0;
4.     sccb_reg_addr <= cur_reg_addr;
5.     sccb_wr_data <= cur_reg_data;
6.     sccb_start   <= 1'b1;
7.     state        <= S_WAIT_WRITE;
8.   end
9. end
10.
11. S_WAIT_WRITE: begin
12.   if (sccb_done) begin
13.     if (cur_last)
14.       state <= S_DONE;
15.     else
16.       state <= S_NEXT_REG;
17.   end
18. end

```

上述代码中，sccb_start 只拉高一个 clk 周期，相当于给 sccb_master 一个启动脉冲。之后状态机等待 sccb_done，而不是固定等待某个时间，这样可以适应不同 SCCB 频率下的传输时间。

② sccb_master 子模块设计

sccb_master 是底层串行总线控制器。它将一个寄存器访问拆成多个状态：起始状态、发

送 8 位数据、等待应答、读数据、发送非应答、停止状态等。发送每一位时，先在 SCL 低电平阶段设置 SDA，再拉高 SCL 保持数据，最后拉低 SCL 准备下一位。

在写寄存器操作中，`phase_idx` 用来记录当前已经发送到第几个阶段：`phase_idx=0` 发送 OV7670 写地址，`phase_idx=1` 发送寄存器地址，`phase_idx=2` 发送寄存器数据。每发送完一个 8 位数据后进入 `ST_DC_SETUP/ST_DC_HIGH1/ST_DC_HIGH2/ST_DC_LOW`，释放 SDA 读取应答位。

```

1. ST_DC_LOW: begin
2.     sio_c <= 1'b0;
3.
4.     if (!op_read) begin
5.         if (phase_idx == 2'd0) begin
6.             phase_idx <= 2'd1;
7.             tx_shift <= reg_addr_l;
8.             bit_cnt <= 3'd7;
9.             state <= ST_TX_SETUP;
10.        end else if (phase_idx == 2'd1) begin
11.            phase_idx <= 2'd2;
12.            tx_shift <= wr_data_l;
13.            bit_cnt <= 3'd7;
14.            state <= ST_TX_SETUP;
15.        end else begin
16.            state <= ST_STOP_A;
17.        end
18.    end
19. end

```

该设计也保留了读操作逻辑。读寄存器时先以写模式发送寄存器地址，然后重新 START，再发送读地址 0x43，最后释放 SDA 接收摄像头返回的数据。虽然本项目主要使用写寄存器完成初始化，但保留读逻辑有利于后续读取摄像头 ID 或调试寄存器状态。

③ ov7670_reg_rom_rgb565 子模块设计

该模块采用 `case(index)` 的方式实现寄存器表。相比把寄存器配置写散在状态机中，单独建立 ROM 更清晰，也便于后续切换分辨率、切换彩条测试或修改颜色配置。调试彩条时只需要修改 ROM 中相关寄存器，而不必改动 SCCB 主控状态机。

例如当前配置中最后一项 0x42=0x00，并将 `last` 置 1，表示寄存器表写入结束。若要打开测试彩条，则需要将相关寄存器配置改为彩条模式；若要回到正常图像采集，则应关闭测试图案，保证摄像头输出真实画面。

④ ov7670_dvp 子模块设计

DVP 子模块最关键的是正确处理 VSYNC、HREF 和 `byte_phase`。如果 VSYNC 判断错误，会导致帧起点错误；如果 HREF 判断错误，会导致行起点或行尾错误；如果 `byte_phase` 没有在帧/行切换时清零，就可能把上一行的残留字节和下一行第一个字节拼成错误像素。

因此代码在 `frame_start`、`line_start` 和 `line_end` 处都对 `first_byte` 和 `byte_phase` 做清零处理。这样即使某一行字节数异常，也不会把错误的半个像素延续到下一行。

```

1. if (href_active_rise) begin
2.     line_start <= 1'b1;
3.     x_cnt <= {X_BITS{1'b0}};

```

```

4.    first_byte    <= 8'h00;
5.    byte_phase    <= 1'b0;
6.    line_has_pixel <= 1'b0;
7. end
8.
9. if (href_active_fall) begin
10.    line_end    <= 1'b1;
11.    x_cnt        <= {X_BITS{1'b0}};
12.    first_byte <= 8'h00;
13.    byte_phase <= 1'b0;
14.
15.    if (line_has_pixel)
16.        y_cnt <= y_cnt + 1'b1;
17. end

```

line_has_pixel 的作用是防止空行也让 y_cnt 增加。只有当前行确实产生过 pixel_valid, HREF 下降沿时才认为这一行有效, 并将 y_cnt 加 1。这对稳定产生 640×480 坐标非常重要。

⑤ to_vdma_axis 子模块设计

AXI 输出模块的设计重点是不能丢失握手规则。摄像头输入像素是连续的, 而 VDMA/FIFO 接收端可能因为内部缓存或总线仲裁短暂拉低 tready。模块通过 can_load_new 判断当前是否可以装载新像素, 只有输出寄存器为空或者上一个像素已经完成握手时, 才允许写入新的 tdata。

同时, 模块利用 pixel_x 和 pixel_y 产生三个关键判断: is_first_pixel 用于 tuser, is_line_last 用于 tlast, is_frame_last 用于 axis_frame_done。这样后级 VDMA 可以根据 tuser 识别帧开始, 根据 tlast 识别行结束, 按照固定行宽写入 DDR。

```

1. wire is_first_pixel;
2. assign is_first_pixel = (pixel_x == 0) && (pixel_y == 0);
3.
4. wire is_line_last;
5. assign is_line_last = (pixel_x == FRAME_WIDTH - 1);
6.
7. wire is_frame_last;
8. assign is_frame_last =
9.    (pixel_x == FRAME_WIDTH - 1) &&
10.    (pixel_y == FRAME_HEIGHT - 1);

```

该模块还输出 axis_sent_pixel_count, 用于统计当前帧已经发送的像素数量。调试时如果 axis_sent_pixel_count 不能接近 FRAME_WIDTH×FRAME_HEIGHT, 说明可能存在像素过滤、tready 背压或 DVP 输入异常。

(4) IP 封装

本次封装以 ov7670_camera_top.v 作为顶层文件, 将 ov7670_sccb_init.v、sccb_master.v、ov7670_reg_rom_rgb565.v、ov7670_dvp.v、to_vdma_axis.v 等文件加入 Vivado IP Packager 的综合和仿真文件集中。封装后的 IP 位于 D:/7670vdmafinal/ip_repo/ov7670_camera_ip, 工程 IP 仓库路径设置为 D:/7670vdmafinal/ip_repo。

计算机硬件类综合性课程设计

表 3.3-2 IP 封装基本信息

封装项	结果
IP 显示名称	OV7670 Camera AXI4-Stream IP
VLNV	xilinx.com:user:ov7670_camera_top:1.0
顶层 HDL	ov7670_camera_top
设计者标识	Li Jiameng
IP 仓库路径	D:/7670vdmafinal/ip_repo
IP 目录	D:/7670vdmafinal/ip_repo/ov7670_camera_ip

封装时需要把 `clk` 和 `cam_pclk` 都定义为时钟接口，但二者作用不同。`clk` 用于 SCCB 初始化、复位延时和 `ov_xclk` 分频；`cam_pclk` 是摄像头返回的像素时钟，DVP 采集和 AXI4-Stream 输出均在该时钟域下工作。因此 `m_axis` 必须关联到 `cam_pclk`，而不是系统 `clk`。若关联错误，Block Design 中连接 VDMA 或 AXI Data FIFO 时可能出现接口检查警告，甚至导致后续上板时视频流握手不稳定。

表 3.3-3 IP 关键参数

参数名	默认值	作用
CLK_FREQ_HZ	100000000	系统时钟频率，用于计算 SCCB 初始化延时、SCCB 分频以及摄像头复位等待时间。
SCCB_FREQ_HZ	100000	SCCB 总线目标频率，默认 100 kHz，满足 OV7670 寄存器配置的低速控制需求。
FRAME_WIDTH	640	有效输出图像宽度，用于判断每行最后一个像素并产生 <code>tlast</code> 。
FRAME_HEIGHT	480	有效输出图像高度，用于判断一帧有效区域并过滤多余数据。
SWAP_BYTES	0	DVP 拼接时是否交换两个字节，便于调试不同 RGB565 字节顺序。

表 3.3-4 IP 封装端口与接口

接口/端口	方向	说明
<code>clk</code>	输入	系统时钟，封装为 <code>clock</code> 接口，用于 SCCB 初始化、复位延时和 <code>ov_xclk</code> 分频。

计算机硬件类综合性课程设计

rst_n	输入	低有效复位，封装为 reset 接口。
cam_pclk	输入	摄像头像素时钟，封装为 clock 接口，并作为 m_axis 视频流接口的关联时钟。
cam_vsync / cam_href / cam_data[7:0]	输入	OV7670 DVP 视频输入信号，分别表示帧同步、行有效和 8 位图像数据。
sccb_sio_c / sccb_sio_d	输出/双向	SCCB 时钟与数据信号，sccb_sio_d 通过 IOBUF 实现三态双向传输。
ov_xclk / ov_reset_n / ov_pwdn	输出	摄像头外部工作时钟、复位和掉电控制信号。
m_axis	Master	AXI4-Stream 视频主接口，包含 tdata、tkeep、tvalid、tready、tlast、tuser。
init_*、axis_*、dbg_*	输出	初始化状态、帧传输状态、溢出标志和调试观察信号。

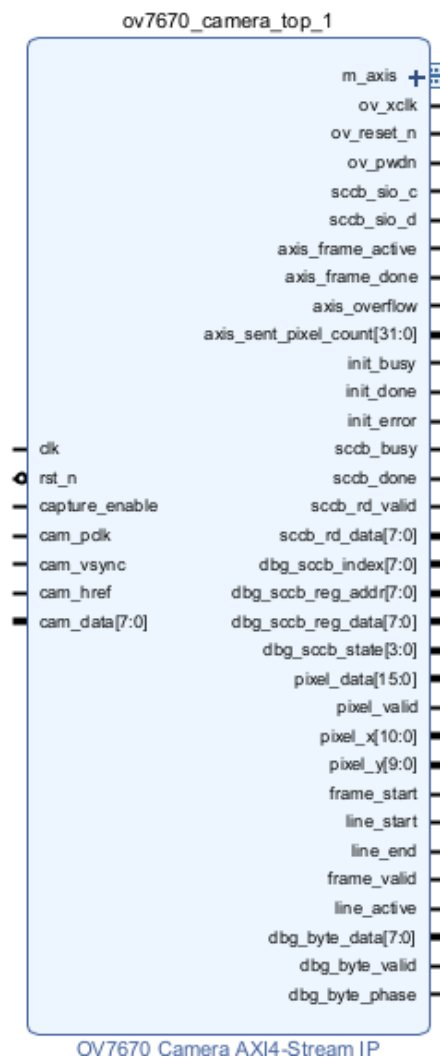


图 3.3-6 ip 封装图

完成封装后，Vivado IP Catalog 能够识别 `xilinx.com:user:ov7670_camera_top:1.0`。后续系统集成时，可以直接从本地 IP 仓库加入该 IP，也可以继续使用现有 block design 中的 `ov7670_camera_top` 实例进行连接验证。由于本模块输出的是标准 AXI4-Stream 视频流，只要 `tdata`、`tkeep`、`tvalid`、`tready`、`tlast`、`tuser` 以及关联时钟设置正确，就可以与 AXI Data FIFO、AXI VDMA 等标准 IP 进行连接。

需要说明的是，在实际工程集成阶段，由于时间限制，后续系统集成同学是在已有工程基础上继续连接和验证，没有完全重新从 IP Catalog 插入封装后的 IP。但从模块结构和接口定义上看，本设计已经按照可复用 IP 的方式整理，满足后续封装和复用要求。

3.3.2 软件设计

本 IP 内部没有设计 AXI-Lite 寄存器，因此软件侧不直接读写 OV7670 IP 内部寄存器。软件设计的重点是系统级控制和帧缓存读取：先加载 bitstream，配置 AXI VDMA 的 S2MM 通道，再等待摄像头初始化完成，最后打开采集使能，使 AXI4-Stream 视频流写入 DDR 帧缓存。

(1) 软件端实现的功能

① 加载硬件 Overlay：在 PYNQ Jupyter 环境中加载 Vivado 导出的 bit 文件和 hwh 文件，使 Python 程序能够访问硬件系统中的 AXI VDMA、GPIO 以及其他外设。

② 配置 VDMA 帧缓存：软件设置 S2MM 通道的帧缓存首地址、水平字节数、垂直行数和 stride。由于本 IP 的 AXI4-Stream 数据宽度为 32 位，一个像素在 DDR 中按 4 字节对齐存放，因此 640 像素宽度对应的水平字节数和 stride 均为 2560。

③ 控制采集启动：系统刚开始运行时保持 `capture_enable` 为 0，避免摄像头还未初始化完成时向 VDMA 写入无效数据。当 `init_done` 为 1 且 `init_error` 为 0 后，再拉高 `capture_enable`，开始向 DDR 写入连续视频帧。

④ 读取和显示图像：软件从 DDR 帧缓存中读取 raw frame，取每个 32 位数据的低 16 位作为 RGB565 像素，然后转换为 RGB888 格式，供 OpenCV、matplotlib 或视频保存程序显示和写入本地文件。

⑤ 录制与稳定性处理：为了降低撕裂和错帧，软件端读取帧时需要尽量避开正在写入的帧缓存，并配合较大的 buffer/frame store 数量。实际调试中，增加缓存数量后，PS 端读取和 VDMA 写入之间的冲突明显减少，图像稳定性提高。

(2) 软件控制流程

软件控制流程可以概括为“先配置缓存，再等待初始化，最后打开采集”。若顺序反过来，摄像头未稳定时产生的无效数据可能进入 DDR，导致开头黑屏、画面错位或保存的视频帧异常。推荐的软件流程如下：

```
1. Overlay("ov7670.bit")
2. OV7670_SetCaptureEnable(0)
3. VDMA_S2MM_Config(frame_addr, width=640, height=480, stride=2560)
4.
5. while init_done == 0:
6.     if init_error == 1:
7.         raise RuntimeError("OV7670 SCCB init failed")
8.
```



```

9. OV7670_SetCaptureEnable(1)
10. while recording:
11.     raw_frame = ReadFrameBuffer()
12.     rgb565 = raw_frame & 0xFFFF
13.     rgb888 = RGB565_to_RGB888(rgb565)
14.     ShowOrSave(rgb888)

```

上述伪代码中，Overlay 加载的是硬件设计导出的 bitstream；VDMA_S2MM_Config 对应 VDMA 写通道配置；ReadFrameBuffer 对应从 DDR 中取出一帧原始数据。由于硬件输出的是 RGB565，而 PC 显示和视频编码通常使用 RGB888 或 BGR888，因此软件端需要完成像素格式转换。

(3) 关键软件参数

表 3.3-5 软件端关键参数

参数	取值	说明
WIDTH	640	输出图像宽度，对应 FRAME_WIDTH。
HEIGHT	480	输出图像高度，对应 FRAME_HEIGHT。
HSIZE	2560	VDMA 每行有效字节数， 640×4 Byte。
STRIDE	2560	DDR 中相邻两行起始地址间隔，当前设置与 HSIZE 相同。
VSIZE	480	VDMA 垂直方向有效行数。
像素有效位	低 16 位	m_axis_tdata[15:0] 为 RGB565，m_axis_tdata[31:16] 补 0。

RGB565 转 RGB888 的核心思想是把 5 位红色、6 位绿色和 5 位蓝色分别扩展到 8 位。例如红色分量来自 pixel[15:11]，绿色分量来自 pixel[10:5]，蓝色分量来自 pixel[4:0]。如果显示时出现红蓝或红绿通道不一致，需要检查硬件寄存器配置、字节顺序以及 Python 端通道拼接顺序是否一致。

3.3.3 模块测试

本模块测试在 Jupyter 上进行，在 Jupyter 上运行软件端录制代码获取完整的录制视频，观测视频是否能正常显示图像，是否有颜色不对，是否有卡顿，错位，撕裂等情况。

下图为视频的截图：



图 3.3-7 板端摄像头实拍测试图

经过调试视频中的撕裂的情况已经减轻了许多，可以满足后续眼部识别算法的需要。

在实际过程中，摄像头模块的难点主要集中在三个方面。第一，OV7670 必须依赖寄存器配置才能输出符合系统要求的图像，SCCB 初始化失败会导致后续完全没有有效图像；第二，DVP 数据是连续字节流，如果 VSYNC、HREF 或 byte_phase 的处理不正确，就会出现像素错位、画面偏移或颜色异常；第三，视频流进入 VDMA 后，PS 端读取帧缓存的时机也会影响显示效果，若读写同一帧缓存，容易出现撕裂或短暂黑屏。因此调试时需要从“寄存器配置—像素拼接—AXI 握手—VDMA 缓存—软件读取”逐级排查。

综上，OV7670 IP 已完成；模块级综合无错误，m_axis 视频流接口已经与 cam_pclk 关联；通过 VDMA 和 PYNQ 软件端读取后，可以完成 OV7670 图像采集、缓存、显示和保存，为后续系统集成提供了稳定的视频输入模块。

3.4 UART-TTS 文字转语音模块的设计（撰写人：康昊轩）

3.4.1 IP 设计

（1）模块基本工作原理介绍

UART 是一种异步收发传输器，其在数据发送时将并行数据转换成串行数据来传输，在数据接收时将接收到的串行数据转换成并行数据，可以实现全双工传输和接收。

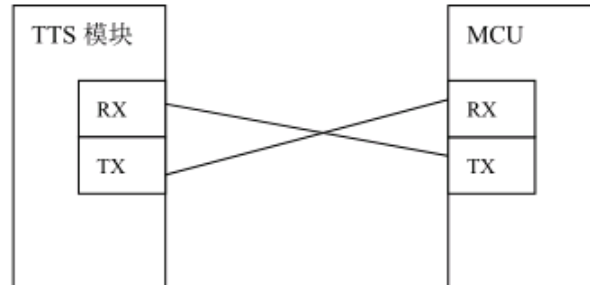


图 1 TTS 模块通讯连接图

图 3.4-1 CN-TTS 语音合成模块实物图

CN-TTS 是一款高集成度的语音合成模块，可实现中文、英文、数字的语音合成；并且支持用户的命令词或提示音的定制需求。CN-TTS 控制方式简单，是通过 TTL 串口发送 GBK 编码的形式，可兼容市面上主流 5V 或 3.3V 单片机。



图 3.4-2 CN-TTS 串口通信协议说明图

通讯标准：UART

波特率：9600bps

起始位：1bit

数据位：8bit

停止位：1bit

校验：无

（2）verilog 代码设计

```

251  // 17.0 msg_data,
252  assign msg_data =
253      (byte_index == 0) ? 8'hBC : // 检 高
254      (byte_index == 1) ? 8'hEC : // 检 低
255      (byte_index == 2) ? 8'hB2 : // 测 高
256      (byte_index == 3) ? 8'hE2 : // 测 低
257      (byte_index == 4) ? 8'hB5 : // 到 高
258      (byte_index == 5) ? 8'hBD : // 到 低
259      (byte_index == 6) ? 8'hD3 : // 用 高
260      (byte_index == 7) ? 8'hC3 : // 用 低
261      (byte_index == 8) ? 8'hBB : // 户 高
262      (byte_index == 9) ? 8'hA7 : // 户 低
263      (byte_index == 10) ? 8'hC6 : // 疲 高
264      (byte_index == 11) ? 8'hA3 : // 疲 低
265      (byte_index == 12) ? 8'hC0 : // 劳 高
266      (byte_index == 13) ? 8'hCD : // 劳 低
267      8'h00;
268  
```

计算机硬件类综合性课程设计

图 3.4-3 UART-TTS 模块发送逻辑代码截图

由于系统设计对该模块的需求十分简单，所以发送内容硬编码在 verilog 端，一次信号输入传送上述内容。

```
1. reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg0; // 映射到地址偏移 0x00
2. reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg1; // 映射到地址偏移 0x04
3. reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg2; // 映射到地址偏移 0x08
4. reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg3; // 映射到地址偏移 0x0C
5. reg start_pulse; // 启动脉冲信号
6. assign slv_reg_wren = axi_wready && S_AXI_WVALID && axi_awready && S_AXI_AWVALID;
```

这行代码表示：只有写地址准备好且有效同时写数据准备好且有效时，写使能才拉高，这是 AXI 全握手条件。

```
1. always @( posedge S_AXI_ACLK ) begin
2.     if ( S_AXI_ARESETN == 1'b0 ) begin
3.         slv_reg0 <= 0; ... start_pulse <= 1'b0;
4.     end else begin
5.         if (slv_reg_wren) begin
6.             case ( axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
7.                 // 即取 axi_awaddr[3:2] 作为索引
8.                 2'h0: // 写 slv_reg0, 并支持WSTRB 字节写掩码
9.                     ...
10.            endcase
11.            if ( axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 2'h0 )
12.                start_pulse <= 1'b1; // 只要向 0x00 偏移写入，就产生单周期高脉冲
13.            else
14.                start_pulse <= 1'b0;
15.        end else begin
16.            start_pulse <= 1'b0;
17.        end
18.    end
19. end
```

axi_awaddr[3:2] 用于区分四个寄存器。slv_reg_wren 为高时，如果地址位 [3:2] 等于 2'h0，则 start_pulse 被赋值为 1'b1，否则为 0。若没有写操作，start_pulse 也是 0。因此该信号是一个只持续一个时钟周期的边沿脉冲。

(3) ip 封装和整体硬件设计

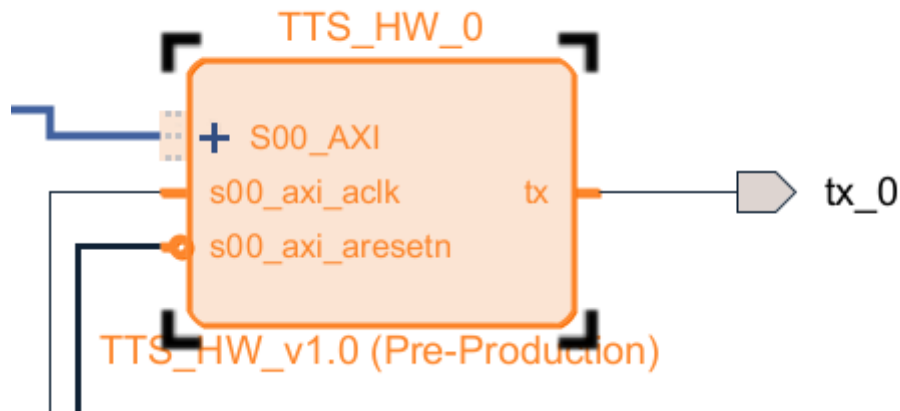


图 3.4-4 UART-TTS IP 封装截图

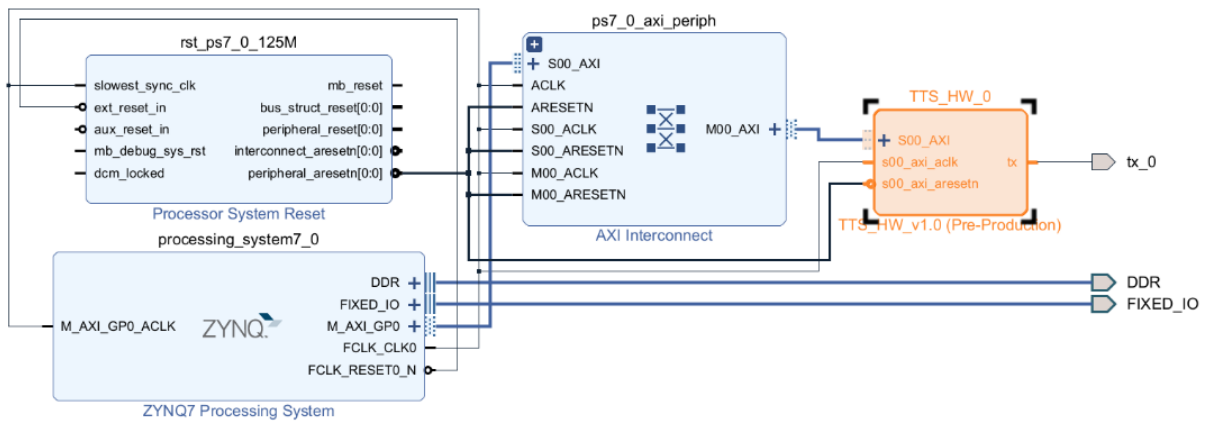


图 3.4-5 UART-TTS 模块 Block Design 集成截图

3.4.2 软件设计

这一部分主要是集成时撰写的，仅展示调试时的基本原理。

```
1. int main() {
2.     init_platform();
3.     print("TTS Start\n");
4.     while (1) {
5.         Xil_Out32(0x43c00000, 0);
6.         usleep(200000);
7.         print("Trigger\n");
8.         sleep(5);
9.     }
10.    cleanup_platform();
11.    return 0;
12. }
```

往 0x43c00000 写入任意数据即可触发 TTS 播报。

3.4.3 模块测试

按照设定好的引脚连线后，在疲劳检测信号触发时，TTS 模块正确播报“检测到用户疲劳”信息。无卡顿，重复触发。

3.5 GPIO RGB 警示灯模块的设计（撰写人：康昊轩）

3.5.1 IP 设计

（1）警示灯 IP 实现的功能

在疲劳驾驶监测系统中，除了屏幕显示预警信息外，还需要一个直观的物理灯光指示器进行强提醒。本模块设计了一个基于有限状态机的警示灯控制 IP，常态下点亮绿灯表示系统运行正常、无疲劳状态；当疲劳检测算法触发预警信号（alert）后，红灯立即点亮并持续 3 秒，随后自动切换回绿灯，等待下一次触发。该 IP 可独立作为 AXI-Lite 外设挂载于 PYNQ 的 PS 端，也可通过外部 GPIO 接收硬件触发信号，具备响应速度快、状态切换清晰的特点。

（2）警示灯工作原理

本 IP 采用有限状态机实现状态管理，共定义两个工作状态：

IDLE（空闲态）：系统处于正常监测状态，绿灯（led_g）点亮，红灯（led_r）熄灭，内部计数器清零。当检测到 alert 信号的上升沿时，状态跳转至 RED 态。

RED（红灯态）：系统处于疲劳告警状态，红灯点亮，绿灯熄灭，内部计数器从零开始累加。当计数值达到预设的时长阈值（RED_TIME）时，状态自动跳转回 IDLE 态。

其中，计数器的时钟频率与系统时钟（100MHz）同步，RED_TIME 参数设定为 300,000,000，对应 3 秒的精确计时。为防止外部输入信号抖动或持续高电平造成重复触发，模块内部对 alert 信号进行两级寄存器同步和边沿检测，确保仅在上升沿的瞬间产生有效的触发脉冲。

表 3.5-1 GPIO RGB 警示灯有限状态机转移表

状态	条件	动作	下一状态
IDLE	alert_posedge=0	绿灯亮，红灯灭，计数器清零	IDLE
IDLE	alert_posedge=1	绿灯亮，红灯灭	RED
RED	cnt < RED_TIME-1	红灯亮，绿灯灭，计数器累加	RED
RED	cnt == RED_TIME-1	红灯亮，绿灯灭，计数器保持	IDLE

（3）警示灯电路设计

本模块的 Verilog 电路设计主要由边沿检测单元、状态机时序逻辑和状态机组合逻辑三部分组成。

① 边沿检测单元

为避免 alert 信号长时间为高导致红灯反复触发或无法退出，采用两级寄存器同步并提取上升沿：

```

1. reg alert_sync, alert_d1;
2. wire alert_posedge;
3.
4. always @(posedge clk or negedge rst_n) begin
5.     if (!rst_n) begin
6.         alert_sync <= 0;
7.         alert_d1  <= 0;
8.     end else begin
9.         alert_sync <= alert;
10.        alert_d1  <= alert_sync;

```



```

11.     end
12. end
13. assign alert_posedge = alert_sync && !alert_d1;

```

该逻辑将外部输入的 alert 信号与系统时钟同步，并在检测到 0→1 跳变时产生一个周期的高电平脉冲 alert_posedge，作为状态转移的精确触发条件。

② 状态机时序逻辑

时序逻辑负责状态寄存器的更新、计数器的累加以及 LED 输出的实时控制：

```

1. always @(posedge clk or negedge rst_n) begin
2.     if (!rst_n) begin
3.         state <= IDLE;
4.         cnt    <= 0;
5.         led_r <= 0;
6.         led_g <= 1;    // 复位后默认绿灯亮
7.     end else begin
8.         state <= next_state;
9.         case (state)
10.            IDLE: begin
11.                led_r <= 0;
12.                led_g <= 1;
13.                cnt    <= 0;
14.            end
15.            RED: begin
16.                led_r <= 1;
17.                led_g <= 0;
18.                if (cnt < RED_TIME - 1)
19.                    cnt <= cnt + 1;
20.                else
21.                    cnt <= 0;    // 计时结束，计数器归零
22.            end
23.            default: begin
24.                led_r <= 0;
25.                led_g <= 1;
26.                cnt    <= 0;
27.            end
28.        endcase
29.    end
30. end

```

在 IDLE 态下，计数器始终清零；在 RED 态下，计数器在每个时钟周期递增，直至达到 RED_TIME-1 后回零，为下一次计时做准备。

③ 状态机组合逻辑

组合逻辑根据当前状态和触发条件决定下一状态的跳转方向：

```

1. always @(*) begin
2.     next_state = state;
3.     case (state)

```

```

4.      IDLE: begin
5.          if (alert_posedge)
6.              next_state = RED;
7.          end
8.      RED: begin
9.          if (cnt == RED_TIME - 1)
10.              next_state = IDLE;
11.          end
12.          default: next_state = IDLE;
13.      endcase
14. end
    
```

该逻辑简洁清晰，确保了状态转移的确定性和实时性。

④ 仿真辅助信号

为便于板级调试和逻辑分析仪观测，模块将当前状态 `state` 和计数器低 16 位 `cnt[15:0]` 分别引出至 `dbg_state` 和 `dbg_cnt` 输出端口，方便在 FPGA 内部抓取波形验证时序。

(4) IP 封装

本模块在 Vivado 中被封装为 AXI-Lite 外设，通过 AXI 总线与 ZYNQ 处理系统连接。PS 端通过对特定基地址进行内存映射写操作，即可产生触发脉冲。IP 内部寄存器映射如下：

表 3.5-2 GPIO RGB 警示灯 IP 寄存器映射

偏移地址	寄存器名	读写属性	功能描述
0x00	TRIGGER	写	写入 bit0=1 时，在内部产生一个等效于 <code>alert</code> 上升沿的触发脉冲，启动 3 秒红灯计时

封装后，IP 对外引脚分配如下表所示：

表 3.5-3 GPIO RGB 警示灯 IP 端口定义

端口名	方向	功能	分配引脚（示例）
<code>clk</code>	input	系统时钟（来自 AXI-Lite 接口）	-
<code>rst_n</code>	input	系统复位，低有效	-
<code>alert</code>	input	外部硬件触发信号（可选）	J17
<code>led_r</code>	output	红灯控制（高电平点亮）	K16
<code>led_g</code>	output	绿灯控制（高电平点亮）	K17
<code>dbg_state</code>	output	调试状态输出	-
<code>dbg_cnt[15:0]</code>	output	调试计数器输出	-

所有触发操作通过 AXI-Lite 写入 TRIGGER 寄存器实现。

3.5.2 软件设计

(1) 寄存器地址映射

在 Vitis SDK 或 PYNQ 的 Linux 环境下，通过内存映射访问 AXI-Lite 外设。根据 Vivado Address Editor 分配的地址，定义如下：

```
1. #define WARNING_LIGHT_BASE_ADDR 0x43C00000
2. #define REG_TRIGGER_OFFSET      0x00 // 写 bit0=1 触发红灯
```

(2) 驱动程序实现

软件驱动逻辑极为精简，主循环中向触发寄存器写入 0x01 产生上升沿脉冲，随后等待 4 秒（红灯持续 3 秒后自动恢复绿灯，预留 1 秒余量以确保下一次触发不会与状态切换冲突）：

```
1. #include "xil_io.h"
2. #include "sleep.h"
3.
4. #define WARNING_LIGHT_BASE_ADDR 0x43C00000
5. #define REG_TRIGGER_OFFSET      0x00
6.
7. int main() {
8.     while (1) {
9.         // 写入 bit0=1，产生触发脉冲（红灯亮 3 秒）
10.        Xil_Out32(WARNING_LIGHT_BASE_ADDR + REG_TRIGGER_OFFSET, 0x01);
11.
12.        // 等待 4 秒，覆盖红灯持续时间和状态恢复
13.        sleep(4);
14.    }
15.    return 0;
16. }
```

该设计将硬件计时（3 秒）与软件延时（4 秒）解耦，即使软件循环存在微小抖动，红灯持续时间始终由硬件精确控制，保证了指示时长的稳定性和可靠性。

3.5.3 模块测试

(1) 硬件平台与测试环境

测试平台为 PYNQ-Z2 开发板（搭载 ZYNQ XC7Z020 芯片），板载两颗 LED（红色和绿色）分别连接至 IP 的 led_r 和 led_g 输出引脚。系统时钟频率为 100MHz。仿真阶段将 RED_TIME 参数临时修改为 1000（对应 10 μ s）以加速功能验证，板级验证时恢复为 300_000_000（对应 3 秒）。

(2) 仿真测试

编写 Testbench 对模块进行功能仿真，测试激励依次施加以下场景：

1. 系统复位后，绿灯输出为高，红灯输出为低，状态机处于 IDLE 态。
2. 施加 alert 上升沿脉冲，状态机立即跳转至 RED 态，红灯点亮，绿灯熄灭，计数器从 0 开始递增。
3. 计数器达到 RED_TIME-1 后，状态机自动返回 IDLE 态，绿灯重新点亮，红灯熄灭。
4. 在 RED 态持续期间再次施加 alert 上升沿，模块不受影响（因边沿检测仅对 IDLE 态有

效，且状态机未设计为在 RED 态响应触发)，保持红灯计时完整。

(3) 板级功能测试

将综合后的比特流下载至板卡，运行测试程序，观测板载 LED 状态：

初始状态：绿灯常亮，红灯熄灭，系统处于待命监测状态。

触发状态：向寄存器写入 0x01 后，绿灯立即熄灭，红灯点亮，持续约 3 秒。

恢复状态：3 秒后红灯自动熄灭，绿灯重新点亮，等待下一次触发。

连续多次触发测试表明，红灯每次均完整保持 3 秒，无异常抖动或提前复位现象。

3.6 SPI-OLED 显示屏模块的设计（撰写人：康昊轩）

3.6.1 IP 设计

（1）OLED 的 SPI 通信协议 IP 实现的功能

本设计在 PYNQ 平台的 PL 端实现了一个轻量级的 AXI-Lite SPI 从机控制 IP，命名为 oled_spi_lite。该 IP 的核心功能是：当 PS 端通过 AXI-Lite 总线向特定寄存器写入数据并置位启动信号后，IP 自动将 1 字节数据按照 SPI 时序以模式 0（CPOL=0，CPHA=0）发送至 OLED 显示屏。IP 内部集成时钟分频器，允许用户通过寄存器配置 SPI 时钟频率，以适应不同 OLED 控制器（如 SSD1309、SSD1306、SH1106）的时序要求。IP 同时管理片选（CS#）、数据/命令（D/C#）和复位（RES#）信号，确保与 OLED 模块的完整通信。

（2）SPI 工作原理

SPI 是一种全双工同步串行通信协议，主设备通过时钟信号控制数据传输，从设备在时钟沿采样或改变数据线。本设计采用 4 线 SPI，接口包括：

spi_clk（SCL）：串行时钟，由主机产生。

spi_mosi（SDA）：主机输出从机输入数据线。

spi_cs（CS#）：片选信号，低电平有效。

spi_dc（D/C#）：数据/命令选择，高电平表示数据，低电平表示命令。

由于 OLED 模块仅需接收数据，无需 MISO 线，因此 IP 仅实现单向传输。SPI 传输模式支持模式 0（CPOL=0，CPHA=0），即空闲时时钟为低电平，在上升沿采样数据，下降沿切换数据。数据帧固定为 8 位，传输顺序从最高位（MSB）开始。

（3）SSD1309 OLED 显示屏工作原理

本设计兼容 SSD1309、SSD1306 及 SH1106 三种主流 OLED 驱动芯片。以 SSD1309 为例，其内部集成 128×64 位显存（GDDRAM），按 8 页（Page0~Page7）组织，每页 128 字节，对应屏幕纵向 8 个像素。显示时，数据字节的每一位对应同一列上连续的 8 个像素。SSD1309 支持多种寻址模式，本设计采用页地址模式，即写入数据后列地址自动递增，页地址保持不变，直到用户重新设置页起始地址。初始化时需通过命令配置对比度、寻址模式、显示开关等参数，之后向显存写入数据即可更新屏幕内容。

（4）SPI 电路设计

① 寄存器映射

IP 通过 AXI-Lite 从接口与 PS 通信，内部定义了 4 个 32 位寄存器，地址偏移及功能如下：

表 3.6-1 SPI-OLED IP 寄存器映射

偏移地址	寄存器名	读写属性	功能描述
0x00	CTRL	写	控制寄存器：bit0 = START（启动发送），bit1 = DC（数据/命令），bit2 = CS_N（片选），bit3 = RES_N（复位）
0x04	TXDATA	写	待发送的 8 位数据，仅低 8 位有效
0x08	CLKDIV	写	时钟分频系数，决定 SCL 频率，值范围 2~65535
0x0C	STATUS	读	状态寄存器：bit0 = BUSY（1 表示正在发送，0 表示空闲）

PS 端通过写上述寄存器控制 IP 行为，通过读取 STATUS 查询发送完成状态。

② 内部逻辑与状态机

IP 内部包含一个有限状态机，负责管理 SPI 发送时序。当 CTRL 寄存器的 START 位被置 1 且 BUSY 为 0 时，状态机启动：

1. **加载阶段：**将 TXDATA 寄存器中的 8 位数据加载到移位寄存器，同时将最高位（bit7）输出到 SDA 线，并置 BUSY=1。
2. **时钟分频：**内部计数器对系统时钟（通常为 125MHz）进行分频，分频值由 CLKDIV 寄存器决定。每经过一个分频周期产生一个 SCL 时钟周期。
3. **位发送：**在 SCL 的每个下降沿，移位寄存器左移一位，将下一位数据更新到 SDA 线；在 SCL 的上升沿，OLED 采样 SDA 线。重复此过程直至 8 位全部发送完毕。
4. **完成：**发送完成后 BUSY 自动清零，状态寄存器的 busy 位变为 0，允许 PS 发起下一次发送。

IP 的 SPI 时钟频率计算公式为：

$$f_{SCL} = \frac{f_{clk}}{CLKDIV}$$

其中 f_{clk} 为系统时钟频率。

(5) IP 封装与硬件连线图

IP 通过 Vivado 封装为 AXI-Lite 外设，挂载在 PYNQ 的 ZYNQ 处理系统（PS）的 GP 接口上，地址由 Vivado 自动分配。PS 端通过内存映射方式访问寄存器，实现软件控制。IP 在 PYNQ Overlay 中被识别为 oled_spi_lite_0，供 Python 驱动调用。

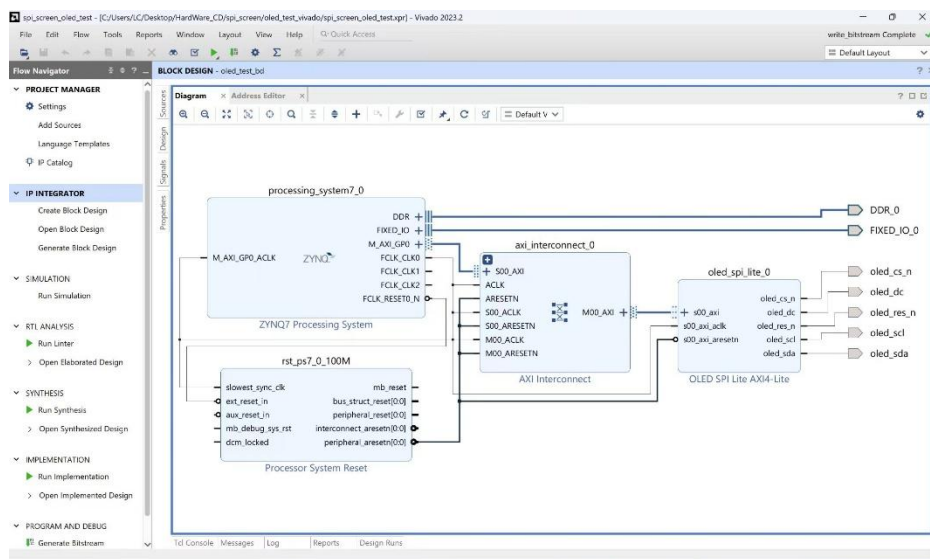


图 3.6-1 SPI-OLED IP 封装与硬件连线图

3.6.2 软件设计

软件驱动基于 Python 编写，运行于 PYNQ 的 Linux 系统，通过 pynq 库操作 Overlay 中的 IP。驱动设计遵循分层结构：底层为寄存器读写封装，中层为 SPI 字节发送函数，上层为 OLED 显示功能函数。

(1) 寄存器访问基础类

定义常量映射寄存器地址：

1. CTRL = 0x00

```
2. TXDATA = 0x04
3. CLKDIV = 0x08
4. STATUS = 0x0C
5. CTRL_START = 1 << 0
6. CTRL_DC = 1 << 1
7. CTRL_CS_N = 1 << 2
8. CTRL_RES_N = 1 << 3
```

OledSpiLite 类封装 IP 操作，通过 ip.write()和 ip.read()方法读写寄存器。

(2) 核心发送函数

write_byte(value, dc)是软件与硬件交互的关键函数，流程如下：

1. 调用 wait_idle()循环读取 STATUS 寄存器，等待 BUSY 位清零，确保 IP 空闲。
2. 设置 DC 和 CS_N 的期望电平（通过修改 CTRL 寄存器的相应位，但不启动发送）。
3. 将待发送数据写入 TXDATA 寄存器。
4. 向 CTRL 寄存器写入 START=1，同时保留 DC、CS_N 等控制位，启动发送。
5. 再次调用 wait_idle()等待发送完成。

该函数实现了单字节的 SPI 发送，后续的 command()和 data()函数均基于它，分别用于发送命令和数据。

(3) 显示缓冲区管理

为减少频繁的 SPI 通信，驱动在 Python 端维护了一个软件显存 self.buf，大小为 width * pages 字节（默认 128×8=1024 字节），对应 OLED 全部像素。所有绘图操作（画点、画线、显示字符等）均先修改缓冲区，最后通过 flush()一次性将缓冲区数据发送至 OLED。

缓冲区索引与像素坐标的映射关系为：

$$idx = (y // 8) * width + x$$

其中 x 为列地址（0~127），y 为行地址（0~63）。y//8 决定页号。

(4) 显示功能实现

① 像素操作

pixel(x, y, on)函数根据坐标计算缓冲区索引和掩码，置位或清零对应位，实现单像素控制。

② 英文字符显示

内置 5×7 点阵字库 FONT5X7，包含大写字母、数字和常用符号。text(x, y, msg)函数将字符串逐字符绘制到缓冲区，每个字符占用 6 列（5 列点阵+1 列空格），行高为 8 像素（与页面高度对齐）。字符绘制通过扫描字库点阵，调用 pixel()实现。

③ 中文字符显示

支持 12×12 点阵中文字库 ZH12，部分常用汉字（如“疲劳”、“窗口”等）以列表存储，每个汉字由 12 个 16 位数据表示，对应 12×12 点阵。zh12(x, y, ch)函数将汉字绘制到缓冲区，与英文字符混排时需注意列偏移。

④ 进度条绘制

hbar(x, y, w, ratio)函数绘制水平进度条，高度为 6 像素，长度 w，填充比例由 ratio（0~1）决定。实现方式为循环调用 pixel 绘制矩形区域。

⑤ 状态信息综合显示

show_status()函数封装了典型应用场景的显示内容，包括系统状态、帧率、疲劳评分、百分比、预测结果等，同时绘制进度条，用于实时反馈疲劳监测系统的运行情况。

(5) 刷新操作

flush()函数将缓冲区内容发送到 OLED。对于 SSD1309/SSD1306, 采用连续写入模式: 先通过命令设置列地址范围和页地址范围(命令 0x21 和 0x22), 然后连续发送所有缓冲区数据。对于 SH1106, 因其列地址不能连续跨越页边界, 采用逐页写入方式: 每页先设置页地址和列起始地址, 再发送该页的 128 字节数据。发送完毕后, 将 CS_N 置高, 结束通信。

(6) 初始化流程

init_display()执行 OLED 初始化:

1. 硬件复位: 拉低 RES_N 并保持, 再拉高, 完成复位。
2. 发送初始化命令序列, 包括关闭显示、设置时钟分频、设置复用率、设置偏移、启用电荷泵、设置寻址模式、设置段重映射、设置 COM 扫描方向、设置对比度、设置预充电周期、设置 VCOM 检测、开启显示等。
3. 清空缓冲区。
4. 刷新显存。
5. 取消片选。

不同控制芯片的命令序列略有差异, 驱动通过 controller 参数区分, 默认支持 SSD1309。

3.6.3 模块测试

(1) 硬件平台与测试环境

测试平台为 PYNQ-Z1 开发板, 搭载 Xilinx ZYNQ XC7Z020 芯片, 操作系统为 Ubuntu, 运行 PYNQ 框架。OLED 模块为 2.42 寸 SSD1309 驱动芯片的 SPI 接口屏 (分辨率 128×64)。PL 端综合了包含 oled_spi_lite IP 的比特流, 通过 Jupyter Notebook 编写测试代码, 调用 Python 驱动进行功能验证。

(2) 功能测试

① 基本显示功能

测试程序依次调用 init_display()初始化屏幕, clear()清空, 然后使用 text()显示英文字符串“STATE:OK”、“FPS: 7.2”等, 使用 zh12()显示中文“疲劳窗口”, 使用 hbar()绘制进度条。实测屏幕能够清晰显示所有内容, 字符无错位、无闪烁, 证明 SPI 通信时序正确, 寄存器读写正常。

② 状态显示测试

调用 show_status(frame=1254, fps=9.2, alert=True, score=774022, ratio=0.36, pred=1, roi=1, eye=1, svm=1), 屏幕显示如下信息:

第一行: 状态“状态: 警报”

第二行: 帧号与帧率“帧率: 9.2 帧数: 1254”

第三行: 疲劳分数“SVM 分数: 774 判: 闭眼”

第四行: 百分比与预测值“比例: 36%”

第五行: ROI、眼睛、SVM 标志“脸: 1 眼: 1 SVM: 1”

底部横线: 窗口中闭眼比例横条

所有信息准确无误, 布局合理, 表明驱动能够灵活组合各类显示元素, 满足实际应用需求。

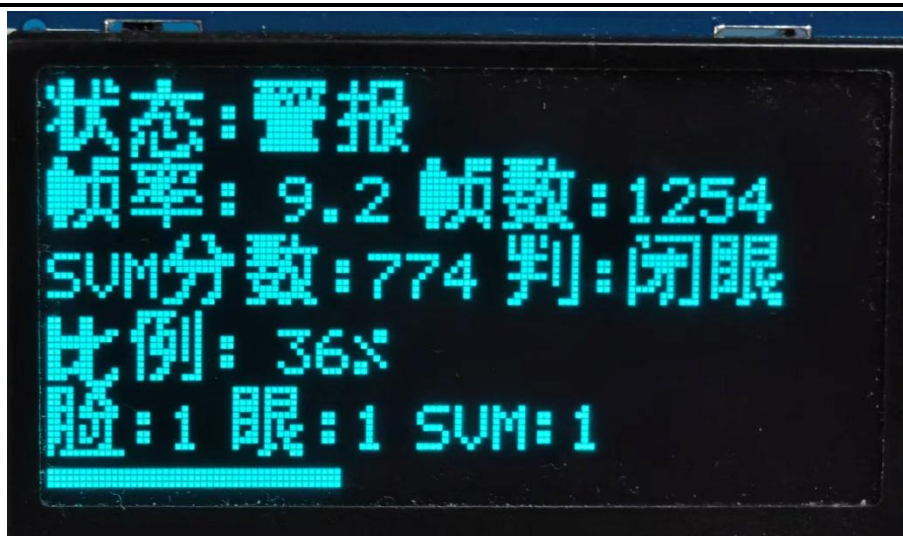


图 3.6-2 OLED 模块板端显示测试图

③ 多控制器兼容性测试

将 OLED 模块更换为 SH1106 驱动芯片的 128×64 屏, 在初始化时指定 `controller="sh1106"`, 驱动自动调整列偏移和刷新策略, 屏幕显示正常, 证实 IP 和驱动具有良好的兼容性。

(3) 性能测试

通过 `time` 模块测量 `write_byte()` 的发送耗时, 在 `CLKDIV=5` 时 (SCL 频率约 25MHz, 但受限于 OLED 实际支持的频率, 本测试设为 5 分频, 约 25MHz, 实测 OLED 可稳定工作), 发送 1024 字节 (一帧) 耗时约 4.8ms, 对应理论帧率约 208FPS, 远高于实时显示需求 (通常 30FPS), 满足系统性能要求。

(4) 测试结论

测试结果表明, 本设计实现的 SPI-OLED 显示模块硬件 IP 与软件驱动配合良好, 能够稳定驱动多种 OLED 控制器, 显示效果清晰, 响应速度快, 可方便地集成到疲劳监测等嵌入式视觉系统中, 具备实际应用价值。

四、系统集成及测试

4.1 硬件系统集成

顶层硬件系统以 Zynq-7000 PS 为控制核心，PL 侧集成摄像头采集、EyeFeature 特征提取、SVM 120D 分类、OLED 显示、RGB LED 和 UART TTS 等模块。摄像头链路负责将 OV7670 的 DVP 图像数据写入 DDR 缓冲，PS 端读取当前帧后更新 ROI；EyeFeature 与 SVM 两个自定义 HLS IP 通过 AXI DMA 传输数据，并通过 AXI-Lite 接收控制参数。

硬件集成时，先完成 OV7670 与 VDMA 的图像通路，再加入 EyeFeature DMA、SVM DMA 与对应 HLS IP，最后接入 OLED、LED、TTS 等输出外设。系统中 SVM 与 EyeFeature 运行在 50 MHz 时钟域，TTS 外设使用独立时钟转换到较高频率，保证各模块在顶层互联中按自身时序约束工作。

表 4.1 硬件集成模块与接口

模块	接口/通路	集成作用
OV7670 采集链路	DVP + AXI VDMA	采集摄像头画面并写入 DDR，供 PS 端读取当前帧
SCCB 摄像头控制	AXI-Lite / SCCB	配置 OV7670 寄存器，完成图像格式、分辨率等初始化
EyeFeature HLS IP	AXI-Lite + AXI DMA/AXIS	接收 ROI 和图像流，输出 8 维眼部图像特征
SVM 120D HLS IP	AXI-Lite + AXI DMA/AXIS	接收 15 帧 120 维窗口，输出 pred 与 score
SPI OLED IP	AXI-Lite + SPI 引脚	显示当前状态、帧率、SVM 分数、闭眼比例和待机提示
GPIO RGB LED IP	AXI-Lite / GPIO	根据疲劳状态控制红绿灯提示
UART TTS IP	AXI-Lite / UART	触发语音播报，实现疲劳警报提醒

图 4.1 为 Vivado 顶层 block design 截图，关注图中 PS、AXI interconnect、VDMA、EyeFeature DMA/SVM DMA、EyeFeature IP、SVM IP 以及 OLED/LED/TTS 外设 IP 的连接关系。该图表明了第三章各模块在硬件层面的集成位置。

计算机硬件类综合性课程设计

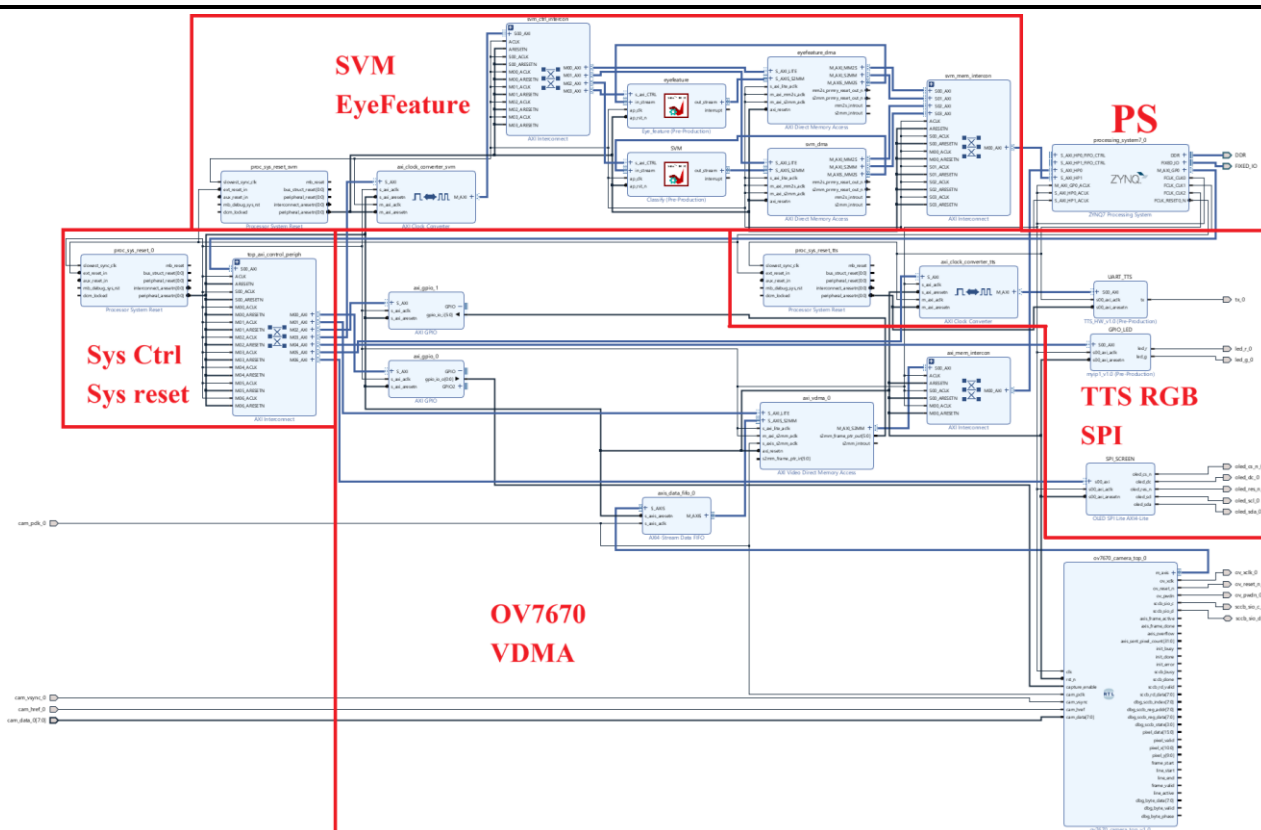
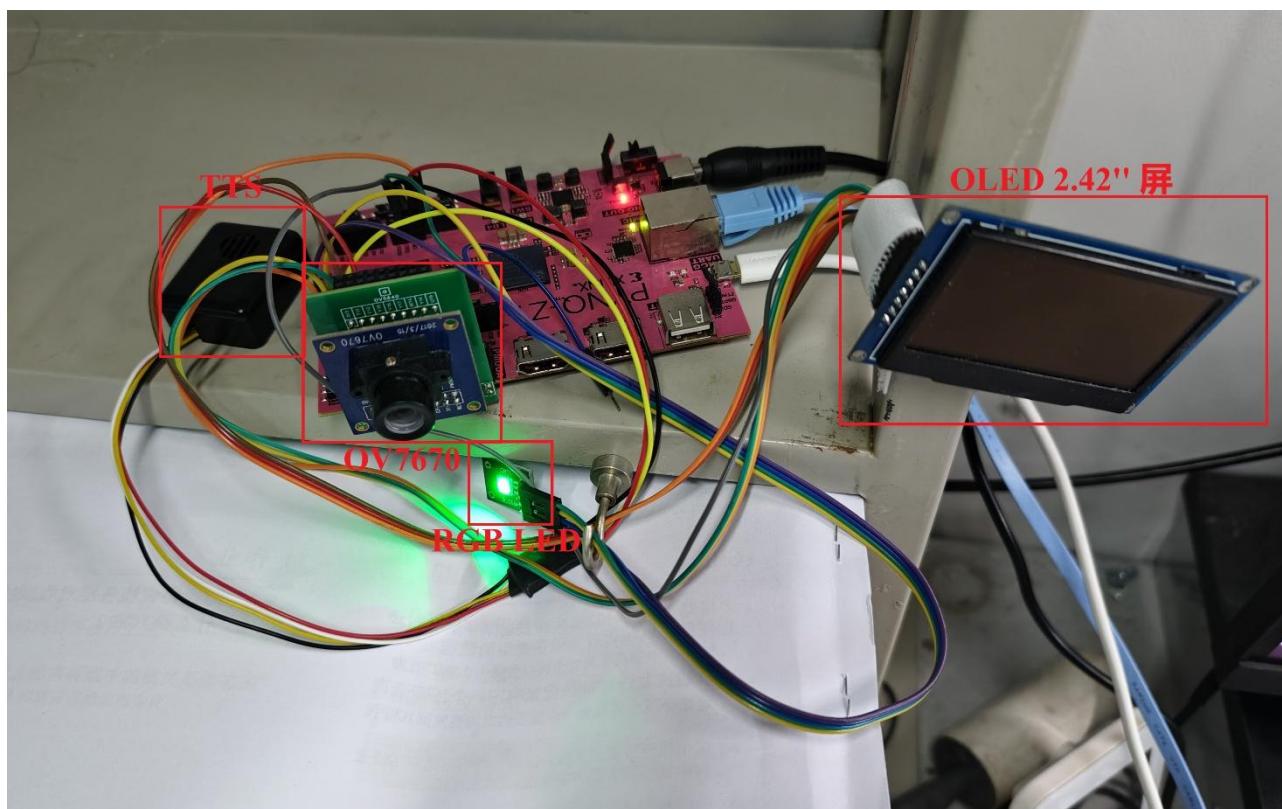


图 4.1 顶层 Block Design 硬件集成图

图 4.2 为最终实物连接照片，展示 PYNQ-Z1、OV7670 摄像头、2.42 寸 OLED、RGB LED、TTS 模块及供电、串口 USB 线和网线连接。实物图能说明系统已经由单个 IP 设计进入整机联调状态。



计算机硬件类综合性课程设计

图 4.2 系统实物连接与最终演示场景图

顶层 Vivado post-implementation 后，整套系统资源占用如表 4.2 所示。结果显示 LUT、BRAM 和 DSP 均未达到器件上限，说明在当前 PYNQ-Z1 的 xc7z020 器件上仍保留一定余量。资源压力主要来自图像采集链路、EyeFeature 图像统计与外设互联，SVM 120D 分类 IP 本身占用较低。

表 4.2 顶层 BD 最终实现资源占用

资源	顶层 post-implementation 占用
LUT	35%
LUTRAM	8%
FF	18%
BRAM	16%
DSP	27%
IO	19%
BUFG	13%

4.2 软件集成与运行流程

PS 端软件以 Python 文件为主脚本。程序启动后首先加载 bit 文件并解析各 IP 名称与基地址，然后初始化摄像头、VDMA、EyeFeature DMA、SVM DMA、OLED、LED 和 TTS。主循环负责读取摄像头帧、复用或更新 dlib ROI、调用 PL 端 EyeFeature 和 SVM，并根据分类结果更新疲劳状态。

软件集成中，dlib 作为低频 ROI 更新线程使用，主要提供人脸框和左右眼 ROI；EyeFeature 与 SVM 作为高频硬件计算链路使用。主循环在获得有效 ROI 后，将 ROI 坐标写入 EyeFeature 的 AXI-Lite 寄存器，并通过 DMA 将当前图像帧送入 EyeFeature；随后把连续 15 帧的 8 维特征组成 120D 窗口，送入 SVM 120D IP 获得 pred 和 score。

正常运行脚本不保存视频帧，仅维护当前帧、ROI、特征滑动窗口、疲劳判决窗口和必要的运行日志，从而避免长时间运行时内存持续增长。若需要调试，可使用 debug 模式输出带 ROI、score、alert 等标注的视频，用于观察定位和分类状态。

图 4.3 为整体系统软件的逻辑图，展示了从程序开始到结束的整个逻辑通路流程。

计算机硬件类综合性课程设计

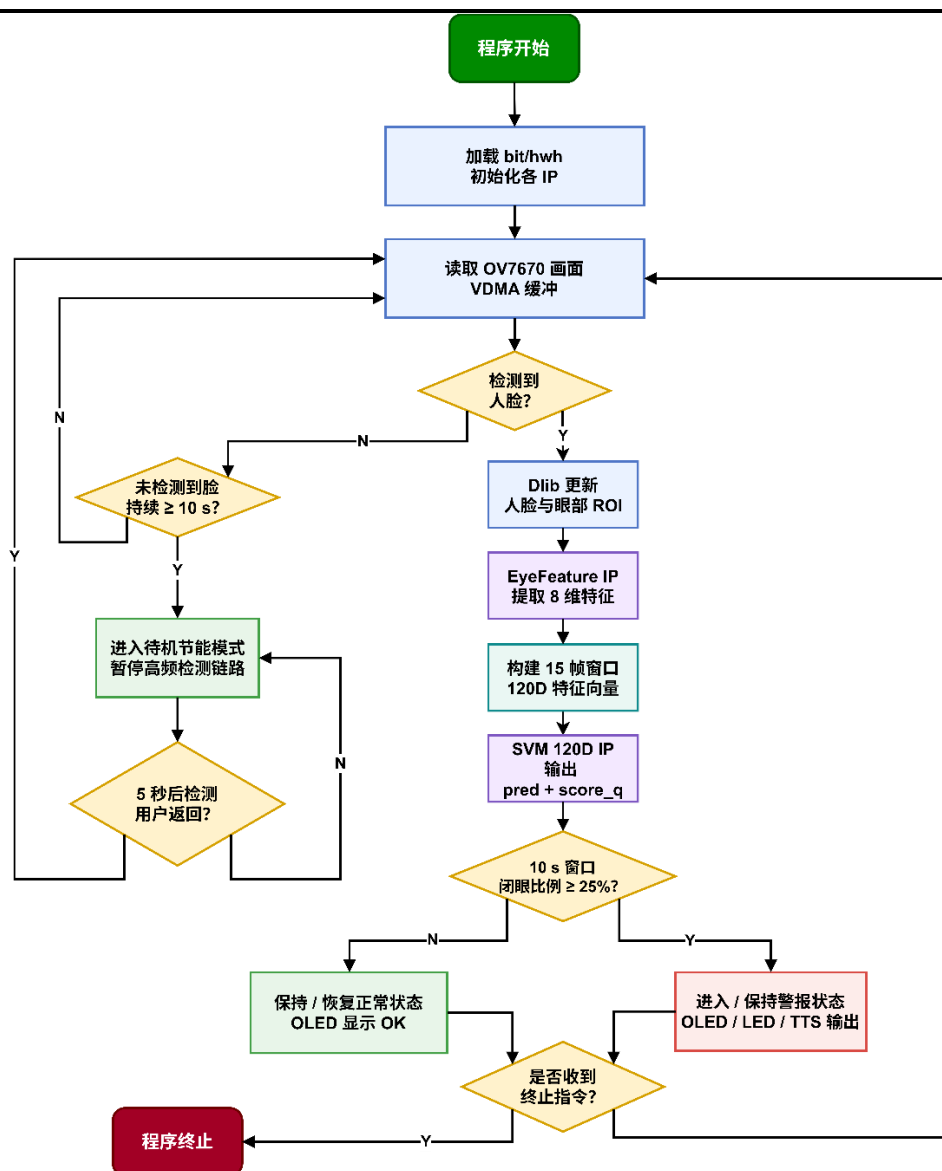


图 4.3 系统软件集成流程图

表 4.3 软件集成数据接口

数据流向	数据内容	软件处理
OV7670/VDMA -> PS	当前摄像头帧	主循环从 DDR 缓冲读取画面，并完成旋转、颜色转换等基础处理
dlib worker -> 主循环	face box、left/right ROI、roi_version	低频更新人脸与眼部 ROI，主循环在有效期内复用 ROI
PS -> EyeFeature IP	图像帧 + ROI 寄存器	通过 AXI-Lite 写 ROI，通过 DMA 发送图像流
EyeFeature IP -> PS	8 维特征、valid、	作为 SVM 输入窗口的单帧特

计算机硬件类综合性课程设计

	roi_version、debug thresholds	征，并用于调试显示
PS -> SVM 120D IP	15 帧 × 8 维 = 120D 特征窗口、threshold_q	通过 DMA 发送窗口，通过 AXI-Lite 写阈值偏置
SVM 120D IP -> PS	pred、score	用于单帧闭眼判断与疲劳滑动窗口统计
PS -> OLED/LED/TTS	状态文本、灯珠控制值、语音触发值	疲劳、正常、待机等状态下输出不同提示

4.3 系统功能测试

系统功能测试按照功能演示表逐项验证，测试对象覆盖图像采集、ROI 定位、EyeFeature 特征输出、SVM 分类、疲劳警报、警报恢复、OLED 状态显示、RGB LED 与 TTS 联动、待机节能和用户返回检测。每项测试均以控制台输出、OLED 显示、debug 视频或实物照片作为现象依据。

表 4.4 系统功能测试结果

测试功能	测试方法	实验现象	结论
实时画面读取与 ROI 定位	运行整体脚本，观察 debug 视频和控制台 roi_valid/roi_version	人脸存在时左右眼 ROI 能跟随眼部区域，roi_version 随 dlib 更新变化	摄像头、VDMA 与 dlib ROI 链路正常
EyeFeature 硬件特征提取	观察 eye_valid、8 维特征、debug_thresholds 和 ROI 标记	睁眼/闭眼时 avg、diff 等结构特征发生变化，eye_valid=1 时输出有效	EyeFeature IP 输出协议和特征计算正常
SVM 120D 分类	输入连续 15 帧特征窗口，观察 pred 与 score	闭眼时 score 趋向正值，睁眼时 score 回到负值或低分区间	SVM 分类链路可用于闭眼判定
疲劳警报触发	连续闭眼或使用闭眼片段触发 10 s 窗口规则	OLED 显示警报状态，RGB LED 进入警报提示，TTS 播报提醒	疲劳警报输出正常
警报恢复	触发警报后持续睁眼或降低闭眼比例	alert 自动清除，OLED 返回 OK/正常状态	警报滞回与恢复规则正常
OLED 状态显示	观察正常、警报、待机三类界面	屏幕显示状态、帧率、帧数、SVM 分数、判	显示内容与运行状态一致

RGB LED 与 TTS 提醒	疲劳事件触发后观察灯珠和播报	定、比例和有效标志 绿/红灯按警报状态变化，TTS 在冷却时间后播报	外设提示与状态机联动正常
待机节能与用户返回检测	用户离开摄像头超过 10 s，随后回到镜头前	OLED 进入待机节能模式，每 5 s 探测一次，人脸返回后恢复检测	待机与恢复流程正常

图 4.4 为板端 OV7670 录制视频的截取画面帧。

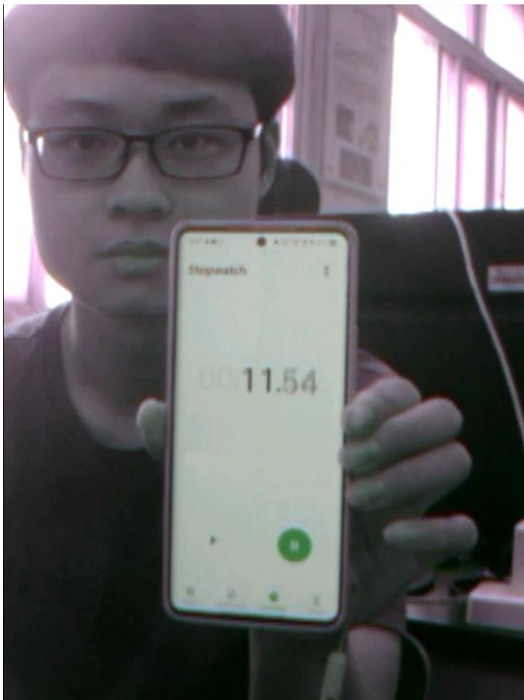


图 4.4 板端实录画面帧

图 4.5 为正常检测状态下的 OLED 照片，能看到 face、eye_valid、svm_run、score、alert 等字段，OLED 上显示“状态: OK/正常”、帧率和 SVM 分数。



计算机硬件类综合性课程设计

图 4.5 正常检测状态 OLED 显示截图

图 4.6 为疲劳警报状态的实物演示截图，展示了 OLED 警报界面、RGB LED 警报状态以及控制台中 `alert=1`、`fatigue ratio` 达到阈值的输出。

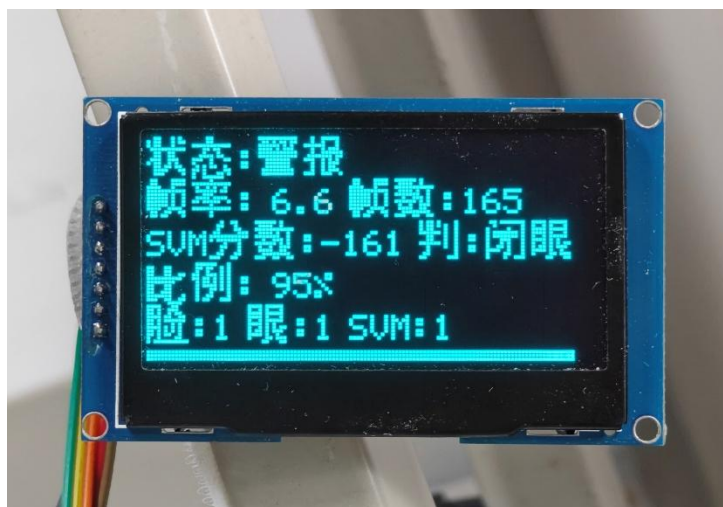


图 4.6 疲劳警报状态实机演示截图

图 4.7 为待机节能模式演示截图，展示用户离开后 OLED 提示“待机节能模式”和“x 秒后检测用户是否返回”。

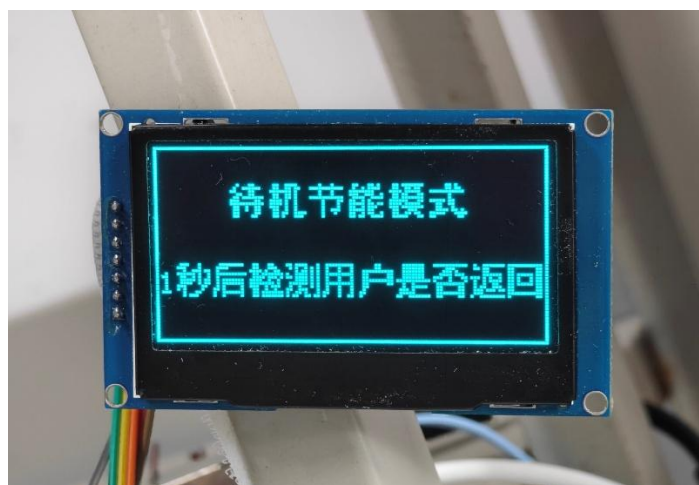


图 4.7 待机节能模式与用户返回检测截图

4.4 性能指标与稳定性测试

性能测试重点验证 PS 端 `dlib` 参数选择、PL 端硬件计算负载以及长时间运行的状态稳定性。由于 `dlib` 人脸关键点检测是 PS 端耗时最大的环节，系统采用较低分辨率和间隔检测方式更新 ROI，而每帧 `EyeFeature` 与 SVM 分类由 PL 端完成，以保证整体主循环能够维持实时运行。

表 4.5 `dlib` 参数与 PS 端性能测试

配置	<code>dlib</code> 分辨率	检测间隔	平均 <code>dlib</code> 耗时	同步循环 FPS
全分辨率逐帧	480×640	每帧	1175.9 ms	0.83

计算机硬件类综合性课程设计

半分辨率逐帧	240×320	每帧	319.8 ms	2.84
1/3 分辨率逐帧	160×213	每帧	155.0 ms	5.00
1/3 分辨率+间隔 3 帧	160×213	每 3 帧	156.2 ms	12.10

由表 4.5 可见，逐帧运行 dlib 会明显降低系统帧率；采用 1/3 分辨率并间隔更新 ROI 后，主循环帧率提升到可用于实时演示的范围。由于 EyeFeature 对 ROI 内灰度结构进行硬件提取，ROI 不必每帧都由 dlib 重新检测，从而减轻 PS 端负载。

表 4.6 疲劳状态机与待机模式参数

规则	当前参数	作用
进入警报	10 s 窗口内有效样本数 ≥ 25 ，闭眼比例 $\geq 25\%$	避免单帧误判直接触发疲劳警报
解除警报	闭眼比例 $\leq 10\%$ ，或连续 4 s 满足 score ≤ 0	引入滞后，避免警报状态频繁抖动
警报输出	OLED 显示警报，RGB LED 闪烁，TTS 播报	提供视觉和语音提醒
TTS 冷却	20 s	避免语音模块连续重复播报
LED 重复周期	约 3.25 s	保持警报可见，同时避免高频刷新
进入待机	连续 10 s 未检测到人脸	无人场景下降低高频检测负载
待机恢复	待机时每 5 s 运行一次 dlib	检测用户返回后恢复正常主循环

表 4.7 系统运行稳定性检查

检查项	实现方式	结果
内存控制	正常模式不保存视频帧，仅维护滑动窗口、状态变量和必要 CSV 日志	长时间运行时避免视频缓存持续增长
DMA 与 IP 重复调用	EyeFeature/SVM 每轮检查 DMA 完成状态和输出长度	数据通路可重复调用，未出现输出长度异常
ROI 复用	dlib 低频更新 roi_version，主循环在有效期内复用 ROI	降低 PS 负载，同时保持眼部区域连续性

计算机硬件类综合性课程设计

输出外设冷却	TTS 设置冷却时间, LED 按周期刷新, OLED 降低刷新频率	减少外设重复触发和 CPU 刷新开销
待机恢复	无人 10 s 进入待机, 每 5 s 探测用户返回	用户返回后可恢复正常检测流程

综上, 系统已经完成从 OV7670 图像采集、PS 端 dlib ROI 更新、PL 端 EyeFeature 特征提取、PL 端 SVM 分类, 到疲劳窗口判决和 OLED/LED/TTS 提醒的完整通路。功能测试和性能测试表明, 系统能够在 PYNQ-Z1 上完成实时疲劳检测演示。

在实机演示时, 出现了 SVM 能正常相应睁闭眼, 但是警报分数阈值设定不合适的情况。在调节 threshold 后能正常触发疲劳警报。后续若继续扩展, 可进一步增强对环境光的适应性、增加自动睁闭眼 threshold 的适应、调节功能。

五、总结

侯景祺：本课程给本人带来了 Arm+FPGA 开发的独特体验，并通过 Arm A9 的性能让我体会到 CPU 计算性能优化的重要性。总体来讲，这是个独特又新奇的开发过程。通过模仿、复现论文，我有机会在 PL 端实现基础的图像处理（如灰度化）和特征提取、并实现线性 SVM 的部署，还实现了和负责其他设备的同学的模块联调。

这个项目，我付出了许多，从一开始 EAR+三分类 SVM 的方案设计（模仿论文实现），到尽全力优化 dlib 性能，到最终转向 EyeFeature+SVM 的方案优化，到最终进行顶层 Block Design 连线、软件联调与编写。期间进行了非常多的试错与调整，比如说发现 dlib 在 PS 端运行延迟过高、原先 EyeFeature 不够鲁棒无法适应强光等。最终成品运作正常，让我十分欣慰。

虽说本科设只是一个相对简单的硬软件合作开发流程，但是对于像我这样的初学者而言，是一个非常有意义和价值的开端。这门课设让我了解和习得了“全栈开发”的基础技能，也是相对于其他学科（例如嵌入式、系统软件开发、深度学习等）的一种全新体验。我认为对于学科领域广阔多元的计算机专业，这种硬软件综合开发是十分有必要去接触和尝试的，正所谓唯有付诸实践才能得知自己是否真正喜爱。

对于课程建议，我本想抱怨 Arm A9 核心性能过于落后，无法适配复杂的逐帧 CV 库运行。但是我了解到 xc7z 系列的 ZYNQ 大多数 SOC 都是配备双核 A9（而且即使有些配备了很大规模的 FPGA 也仍是这样），那么可以大概推测出 PS 性能也许对这种设备没有那么重要，或者说应该采取更多的性能优化而非去选择有更强 CPU 性能的设备。毕竟对于边缘设备而言，能效才是最重要的，开发者应该尽量利用自己的水平来尽量优化计算，发掘硬件的最大潜力。

总而言之，感谢老师的指导，带我们体验这种“全栈开发”。我对课程设备没有明确的优化建议，不过我推荐老师可以更多地引导下一届及以后的学弟学妹们多尝试进行论文复现、CV 模型部署于 PL 端等非同寻常的工作，而非局限于传统协议的开发。这样更能调动创新性、让热爱计算机视觉/机器学习等领域的同学体验到边缘设备部署模型的乐趣。

李佳萌：在本次课程设计中，我主要负责的是 OV7670 摄像头采集 IP 的设计、封装以及与后续 VDMA、PS 端读取显示相关的部分。我的工作重点不是直接做图像识别，而是让摄像头能够稳定输出图像数据，并把 OV7670 的 DVP 并行数据转换成后续系统可以接收的 AXI4-Stream 视频流。简单来说，就是先让摄像头“能出图”，再让图像数据能够比较稳定地进入 DDR，供后续软件端读取、显示和保存。

一开始做这个模块的时候，我以为摄像头部分只是把几个信号接出来，然后按照时序采集数据就可以了。但真正开始做之后才发现，OV7670 并不是上电后就能直接输出我们想要的图像，它必须先通过 SCCB 总线写入一大串寄存器配置。这个过程比我想象中麻烦很多，尤其是寄存器配置、RGB565 输出格式、VSYNC 和 HREF 的时序关系，都需要一点一点查资料、看代码、上板测试。很多时候代码看起来没有明显错误，但是板子上就是没有正确图像，这种情况其实最难排查。

在硬件设计中，我先完成了 SCCB 初始化部分。该部分主要负责给 OV7670 提供 ov_xclk，同时控制 ov_reset_n 和 ov_pwrn，使摄像头能够按照正确顺序复位和启动。之后通过 sccb_master 模块按照 ROM 中保存的寄存器表逐项写入配置，使摄像头输出 RGB565 格式的数据。这个模块调试的时候，我比较关注 init_done、init_error、sccb_busy、sccb_done 等信号，因为这些信号可以判断寄存器到底有没有写完。如果 SCCB 初始化失败，后面的 DVP 采集和 VDMA 读取其实都没有意义。

完成初始化之后,我又继续做了 DVP 数据采集部分。OV7670 输出的是 8 位并行数据,而 RGB565 一个像素需要两个字节,所以我在模块中使用 `byte_phase` 来区分当前采集的是第一个字节还是第二个字节。第一个字节先保存在 `first_byte` 中,第二个字节到来时再拼接成一个 16 位的 `pixel_data`,同时产生 `pixel_valid`。这个地方看起来很简单,但实际调试时如果 `VSYNC`、`HREF` 或 `byte_phase` 的处理稍微不对,就会出现画面错位、颜色异常、图像撕裂等问题。有时候画面不是完全黑屏,而是能看到一点图像但位置不对,这种情况反而更难判断问题到底出在寄存器、采集时序还是后级缓存。

后面为了让图像数据能进入 VDMA,我又把采集到的 RGB565 像素封装成 AXI4-Stream 格式输出。这里需要产生 `tdata`、`tvalid`、`tready`、`tlast`、`tuser` 等信号。其中 `tuser` 用来标记一帧的第一个像素,`tlast` 用来标记一行的最后一个像素。刚开始我对 AXI4-Stream 的握手机制理解不够深,只知道 `tvalid` 表示数据有效,但后来调试过程中才真正理解,只有 `tvalid` 和 `tready` 同时有效,数据才算真正传输出去。如果下游 VDMA 或 FIFO 没有准备好,摄像头又持续输出像素,就可能出现丢像素或者画面撕裂,所以我在模块中增加了 `axis_overflow` 这样的调试信号,用来判断是否存在背压问题。

整个过程中最折磨人的地方其实不是写代码,而是反复验证。每次修改 Verilog 后都需要重新综合、实现、生成 bit 文件,等待时间比较长。有时候只是改了一个很小的地方,也要重新等一遍流程。更麻烦的是,摄像头模块不像普通的通信模块那样可以直接用串口打印结果,也不像纯软件程序那样可以一步一步调试。它涉及摄像头寄存器、DVP 时序、AXI4-Stream 握手、VDMA 缓存、PS 端读取等多个环节,只要其中一个地方有问题,最终看到的现象可能都是黑屏、花屏或者撕裂,很难一下子定位。

在调试过程中,我也走了不少弯路。前期尝试过不同的采集和显示方式,也遇到过颜色不对、画面错位、视频像是被加速、录制时开头黑屏等问题。后来通过不断调整 VDMA 参数、缓存数量以及 Python 端读取方式,画面撕裂的情况明显减轻。虽然最后的效果不敢说非常完美,但能够在 Jupyter 端读取到摄像头图像,并且可以完成视频显示和保存,这对我来说已经是一个比较大的突破。尤其是在画面终于稳定显示出来的时候,确实有一种前面反复调试没有白费的感觉。

这次课程设计让我学到了很多之前没有接触过的内容。以前我对 FPGA 的理解更多停留在写模块、连线 and 仿真上,但这次真正感受到一个硬件模块要跑起来,需要考虑的不只是代码本身,还包括时钟域、复位顺序、接口协议、缓存机制和软件读取方式。通过这个项目,我对 SCCB、DVP、AXI4-Stream、VDMA、PYNQ 以及 ILA 调试都有了更具体的认识,也明白了硬件调试很多时候不是一次就能成功,而是要根据现象不断缩小问题范围。

当然,这个模块还有很多可以继续改进的地方。比如摄像头图像的稳定性还可以进一步提高,帧缓存读取时机也可以设计得更完善,软件端和硬件端之间的配合还可以更规范。如果以后有更多时间,我希望能继续完善这个摄像头采集链路,让它在不同场景下都能更稳定地工作。

对于这门课程,我个人觉得难度还是比较大的,尤其是像摄像头这种模块,刚开始接触时确实会有些无从下手。很多问题资料上不会直接告诉答案,只能自己查资料、看波形、上板验证,一点一点试出来。虽然过程比较痛苦,但也正是因为这样,我才真正学到了很多工程中会遇到的问题。最后不管成绩如何,我觉得这次课程设计对我来说还是很有收获的,也让我更加明白了硬件系统从“能写出来”到“能稳定运行”之间还有很长一段距离。

康昊轩: 最后的总结,不提技术相关的问题,只说心里话。

无论过程怎样,计算机硬件课设的任务到今天总算是告一段落了。这是我第一次参与复杂的多人合作的与硬件相关的课程设计。虽然我在其中负责的并不是复杂的模块(SPI/GPIO/UART),但我的工作量也是十分到位的,从零开始学习 vivado,vitis 的软件操作,

计算机硬件类综合性课程设计

我在嵌入式也实现过 TTS 播报的类似功能，但是相较于嵌入式对硬件操作只需要简单调用别人写好的库函数，vivado 实现同样功能的过程十分繁琐。硬件端 verilog 代码撰写、tb 代码撰写，波形图调试，ila 观察，硬件连线，axi 设计等，软件端 vitis 代码撰写等，单拎出来一部都是十分恼人的，我可以毫不保留的说，硬件课设是我遇到过最难的课设，没有之一。但是在这里我可以很自豪地说，尽管我不是很聪明，但是所有部分我都完成了。首先，我要感谢我自己。

其次，我要感谢我的同组成员侯景祺和李佳萌对我的协助，没有他们的帮助，我的进度推进举步维艰，更别谈整合了。最后，非常感谢我的教师王宗侠老师对我们整组的协助，无论是从最开始的选题设计，再到实现过程中的技术指导，再到最后的答辩和汇报，都少不了老师的帮助。感谢你们的帮助，整个过程中我都受益匪浅。

附录

参考文献

Soukupova T., Cech J. Real-Time Eye Blink Detection using Facial Landmarks. CVWW, 2016.

Kazemi V., Sullivan J. One Millisecond Face Alignment with an Ensemble of Regression Trees. CVPR, 2014.

Dasgupta A., George A., Happy S. L., Routray A. A Vision Based System for Monitoring the Loss of Attention in Automotive Drivers. IEEE T-ITS / arXiv, 2015.

Souto Maior C. B., Moura M. C., Santana J. M. M. de, Nascimento L. M. do, Macedo J. B., Lins I. D., Droguett E. L. Real-time SVM Classification for Drowsiness Detection Using Eye Aspect Ratio. Proceedings of Probabilistic Safety Assessment and Management PSAM 14, 2018.

Vidhyapathi C. M., Maheshwar Reddy M., Nikhil Reddy T., Raj A. N. J., Kathirvelan J. Hardware Acceleration of SVM Classifier Using Zynq SoC FPGA. International Journal of Innovative Technology and Exploring Engineering, 8(12):2280-2288, 2019.

计算机硬件类综合性课程设计